

UNIVERSITY OF CALIFORNIA
SANTA CRUZ

**SHARKLINQ: INTEGRATING MULTIMODAL MODELING AND UI
DEVELOPMENT TO ADVANCE SHARK RE-IDENTIFICATION**

A thesis submitted in partial satisfaction of the
requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE AND ENGINEERING

by

Fabrice Kurmann

June 2026

The Thesis of Fabrice Kurmann
is approved:

Professor Lise Getoor, Chair

Professor Leilani Gilpin

Doctor Connor Pryor

Peter Biehl
Vice Provost and Dean of Graduate Studies

Copyright © by
Fabrice Kurmann
2026

Table of Contents

List of Figures	iv
List of Tables	v
Abstract	vii
Acknowledgments	ix
1 Introduction	1
2 Related Work	5
2.1 Image-Based ReID	5
2.2 Multimodal Approaches	7
2.3 ReID Systems and Tools	8
3 Our Modeling Approach	10
3.1 Foundation Models	11
3.2 Fine-Tuning	12
3.3 Multimodal Approaches	17
3.4 Retrieval Algorithms	23
4 Empirical Evaluation	26
4.1 Dataset	26
4.2 Foundation Models and Fine-Tuning	28
4.3 Multimodal Approaches	30
4.4 Retrieval Algorithms	31
4.5 Data Quality	32
5 Shark Matcher	36
5.1 Architecture	37
5.2 Impact	45
6 Conclusion	47
6.1 Future Work	48
Bibliography	50

List of Figures

1	Example of intra-class image variance between underwater and above water (topshot) photographs.	2
2	The SHARKLINQ model pipeline showing embeddings generation and retrieval of matching individuals.	4
3	Visual examples showing the set of augmentations applied.	17
4	A visual comparison between NN vs. NP retrieval. Each image maps to a point in embedding space. NN retrieval compares image embeddings while NP retrieval builds a single prototype for each individual.	25
5	Number of sharks with counts of matched images showing a long-tailed class distribution (left). Number of images by year (right). . .	27
6	Number of images by year in IMAGE REMOVAL selection (left) and SHARK REMOVAL (right); IMAGE REMOVAL evaluates sharks spanning the full temporal range of the dataset while SHARK REMOVAL evaluates performance on newer images.	28
7	Overview of SHARKLINQ’s dual repository structure.	38
8	Overview of the SHARK MATCHER data architecture, showing the relationship between shark entries, associated images, and associated objects.	39
9	The shark profile page displays a shark entry’s properties, associated images, and associated objects.	40
10	The image upload page during a bulk upload of images.	41
11	The search page shows results grouped by shark entry with key properties and an image gallery for each result.	42
12	The match page displays the query shark (left) and an ordered list of match suggestions (right).	43
13	The merge page displays columns for the sharks to be merged (left) and a preview of the result (right).	44

List of Tables

1	Comparison of the IMAGE REMOVAL and SHARK REMOVAL test sets. The two approaches show a similar representation associated image count and a quality distribution.	27
2	IMAGE REMOVAL retrieval accuracy comparison for baseline and fine-tuned image models by loss function (Triplet vs. ArcFace). . .	29
3	SHARK REMOVAL retrieval accuracy comparison for baseline and fine-tuned image models by loss function (Triplet vs. ArcFace). . .	29
4	IMAGE REMOVAL retrieval accuracy comparison for image-only and multimodal approaches for best performing ViT and MiewID models. ME is MULTIMODAL EMBEDDINGS. SR is SYMBOLIC RE-RANKING.	30
5	SHARK REMOVAL retrieval accuracy comparison for image-only and multimodal approaches for best performing ViT and MiewID models. ME is MULTIMODAL EMBEDDINGS. SR is SYMBOLIC RE-RANKING.	31
6	IMAGE REMOVAL accuracy comparison for NN and NP retrieval with best performing ViT and MiewID models.	32
7	SHARK REMOVAL accuracy comparison for NN and NP retrieval with best performing ViT and MiewID models.	32
8	Number of queries across the IMAGE REMOVAL and SHARK REMOVAL test sets where the correct match lies within the first 50 results.	33
9	IMAGE REMOVAL retrieval accuracy of the fine-tuned MiewID image-only model, bucketing results by query shark frequency and image quality.	33
10	SHARK REMOVAL retrieval accuracy of the fine-tuned MiewID image-only model, bucketing results by query shark frequency and image quality.	34
11	INITIAL and FULL snapshot comparison showing the amount of additional data added and extent of labeling updates between the two. .	34

12	IMAGE REMOVAL retrieval accuracy comparison for image-only and multimodal models trained on the INITIAL and FULL dataset snapshots.	35
13	SHARK REMOVAL retrieval accuracy comparison for image-only and multimodal models trained on the INITIAL and FULL dataset snapshots.	35

Abstract

SharkLinq: Integrating Multimodal Modeling and UI Development to Advance
Shark Re-Identification

by

Fabrice Kurmann

Wildlife dataset curation and re-identification (ReID) centers on mapping new images to an existing set of known individuals and are critical tasks for ecology, conservation, and the study of populations. Traditionally, domain experts have performed this task manually, but identifying individuals from images is both challenging and labor-intensive, complicated by long-tailed class distributions, high intra-class variance, and noisy datasets. To address these challenges, we present SHARKLINQ, an end-to-end system that integrates multimodal modeling and retrieval techniques with SHARK MATCHER, a human-in-the-loop tool for model-assisted ReID and general database management.

We detail a parameter-efficient framework for fine-tuning computer vision backbones, specifically evaluating Vision Transformers (ViT) [13] and MiewID [30] (EfficientNetV2) [35]. To leverage heterogeneous supplementary data, we introduce a weighted late fusion approach that constructs multimodal embeddings, combining visual features with structured metadata. Additionally, we optimize retrieval techniques to maximize representation stability, enforcement of metadata constraints, and retrieval accuracy. Empirical evaluations on a sharkfin dataset developed and maintained by Block Lab at Stanford demonstrate that combining a fine-tuned image model with multimodal embeddings and optimized retrieval approach achieves a 90% Hits@1 accuracy and a mean average precision of 0.924.

These modeling advancements are deployed through SHARK MATCHER, a tool which streamlines data curation by presenting model ReID suggestions in a UI that supports expert verification and multi-user collaboration. Ultimately, this integration of a multimodal model with an accompanying interface significantly accelerates data curation, improves overall dataset quality, and serves as a framework

for model-assisted ReID. We release both our the SHARKLINQ model framework and SHARK MATCHER repositories with code and documentation to support future research and adoption within the ecology community.

Acknowledgments

This project has been endlessly engaging and fulfilling, all while serving as an excellent source of new challenges, each of which has been invaluable to my graduate education and research. To this end, I am extremely grateful to my advisor, Professor Lise Getoor, who gave me the opportunity to join her lab and this project as an undergraduate and has been supporting me ever since. Lise has taught me countless valuable and universally relevant lessons that I will take with me wherever I go. She has also been a constant advocate for both my research and work as a TA, which I am extremely grateful for.

I would also like to extend my deepest gratitude to Connor Pryor, Charles Dickens, and Eriq Augustine, my colleagues at the LINQS lab, all of whom made significant contributions to this project. Connor, Charles, and Eriq devoted countless hours to helping me with everything from model training, to building the Shark Matcher app, to preparing presentations and writing papers. Their patience and commitment to mentoring me, despite requiring far more time and effort than just doing the work themselves, has been a shaping lesson in dedication and selflessness. I hope to one day pass this devotion on to a mentee of my own.

This project would not have gotten started without the Block Lab initiating the collaboration. I'm extremely grateful to Alexandra DiGiacomo and Samantha Andrzejczek and Professor Barbara Block, who have been fantastic collaborators and provided invaluable ideas, insights, and feedback. The interactions with users and getting to learn about their perspectives and workflows has been a formative aspect of this project, challenging me in new ways and teaching me presentation and communication skills I'm thrilled to have acquired.

Finally, thank you very much to my thesis reading committee: Professor Lise Getoor, Professor Leilani Giplin, and Doctor Connor Pryor. I greatly value your expertise, time, and feedback.

Portions of this thesis were supported by National Science Foundation grant CCF-2023495.

1 Introduction

Individual animal re-identification (ReID) is a foundational task in ecological monitoring and conservation, enabling the tracking of population dynamics, migration patterns, and behavior [7, 8, 10, 17, 19, 25–27, 30, 31, 36, 37, 39, 42]. Historically, longitudinal population monitoring relied on manual methods to identify individuals from photos by comparing physical biomarkers such as discriminative body parts, pigmentation patterns, and epibiotic scarring. In this work, we specifically examine the problem of white shark ReID from dorsal fin images extracted from video [1]. We collaborated with the Block Lab at Stanford University’s Hopkins Marine Station to understand their existing workflows and requirements of a system for automated ReID and dataset curation. We then worked with their extensive sharkfin dataset of approximately 4000 images, spanning over 20 years, and representing 1376 individual sharks, to develop a machine learning based ReID framework with human-in-the-loop verification.

Working with a constantly evolving, real-world shark dataset presents substantial challenges: due to the vastness of their habitat, limited visibility, and rarity of re-encountering the same individual, white shark sightings are rare and photographic conditions are often poor due to murky water, uneven lighting, and the difficulty of filming fast-moving sharks [29]. As a result of such constraints, many wildlife image datasets across species feature similar challenges: they are often small-scale and contain images with high variance in quality and background that must be labeled using only subtle differences in animal shape and coloration. Noisy data is common, with inconsistencies that include mislabeled images, duplicate classes that represent the same individual, and inconsistent availability and reliability of accompanying metadata. Additionally, wildlife datasets generally contain large numbers of unique classes, often containing only sparse amounts of data per individual. Datasets are frequently characterized by long-tailed, uneven distributions; in the case of the Block Lab’s sharkfin dataset and its 1376 shark classes, 1042 individuals contain 3 or fewer matched images while 76 sharks have been photographed on 10 or more separate occasions. Lastly, wildlife datasets frequently cover large time

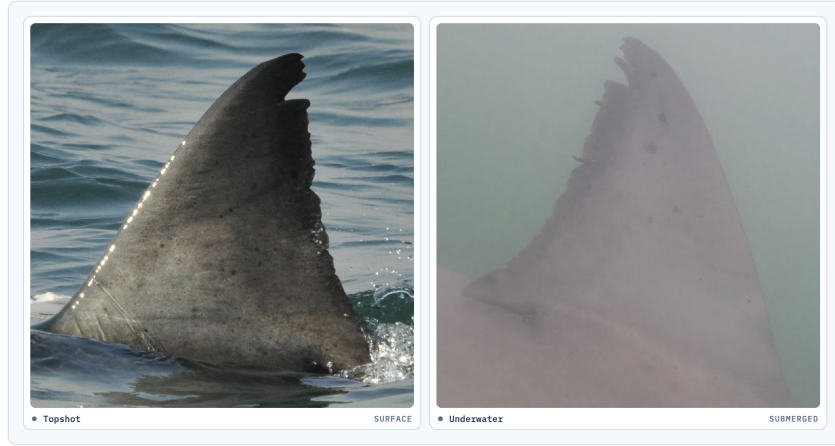


Figure 1. Example of intra-class image variance between underwater and above water (top-shot) photographs.

spans, meaning that matching must take into consideration the natural evolution of the animals. Manual curation of such data is labor intensive, time-consuming, and difficult to scale due to the subtlety of biomarkers, the massive search space of individuals, and the reliance on highly skilled domain experts.

Automated animal ReID represents a critical frontier at the intersection of machine learning, computer vision, and ecology, offering a scalable alternative to manual data labeling. Advancements in modeling have enabled the transition from manual analysis to increasingly accurate automated biometric identification pipelines. Modern ReID frameworks approach the problem as a fine-grained image retrieval task, mapping images into a high-dimensional latent space where embedding distance represents biometric similarity. This task has received significant attention with previous work pioneering numerous methods, most recently applying deep neural network architectures and post-training techniques to build models that can recognize and identify individuals based on specific biomarkers across multiple different species. We build on these efforts and develop modeling and retrieval techniques to improve ReID accuracy.

Many wildlife datasets, including that of the Block Lab contain additional data as a supplement to the raw images. Information such as capture time and location, notes on image quality, and observed biological attributes of the animal, including

length and sex, is highly correlated with an individual’s identity and can greatly aid in ReID. We evaluate the effect on ReID performance when incorporating this metadata into our pipeline, moving from image-only to a multimodal approach.

To allow biology researchers to view and verify labeling suggestions from the trained model, a user interface (UI) is required. Working closely with our collaborators at the Block Lab, we reviewed their data intake and labeling process and considered how a matching interface could be combined to also offer a digital catalog, addressing both the task of streamlining labeling and providing a faster and easier way to view and manage their full dataset. The resulting tool, SHARK MATCHER, leverages our ReID model through a web-based UI, allowing biologists to streamline their data processing while maintaining full control over the final labeling decisions. It also supports general dataset exploration and management workflows and enables collaboration in dataset curation. Our work culminates in SHARKLINK, an end-to-end system which combines a model framework to train custom-trained ReID models and a set of optimized retrieval algorithms with SHARK MATCHER, a research-grade tool designed for automated ReID and wildlife catalog management.

Our contributions are: (1) We present a methodology for efficiently training a neural model by fine-tuning state-of-the-art computer vision backbones via Low-Rank Adaptation (LoRA) and incorporating heterogeneous metadata for multimodal modeling; (2) We develop a retrieval technique that leverages the neural model’s multimodal embeddings with metadata features used as symbolic constraints to maximize ReID accuracy; (3) We pair our model framework with SHARK MATCHER, a tool for automated ReID and dataset management, built in close collaboration with marine biologists at Stanford’s Block Lab. These components result in a model-assisted, ReID workflow which we show has led to significant speedups in data intake, improvements in labeling accuracy, and increased collaboration and sharing of data among biological researchers.

This thesis begins with an extensive review of the ReID literature spanning classical computer vision techniques to the most recent multimodal deep learning methods in Section 2. We build on this foundation and explain in detail our approach in model architecture and fine-tuning, incorporating multimodal data, and retrieval in

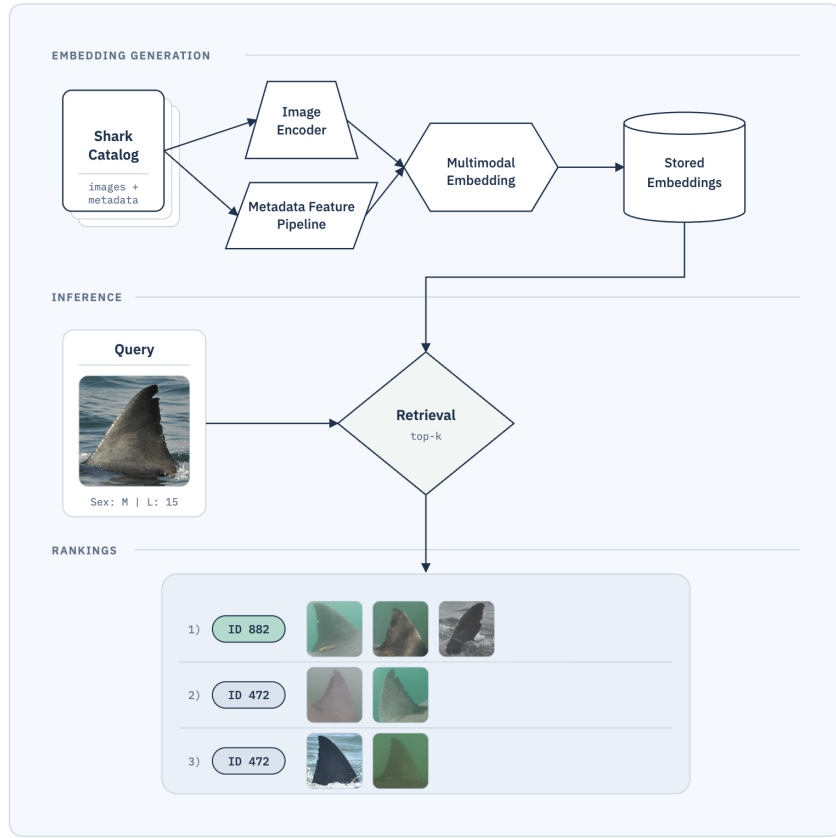


Figure 2. The SHARKLINQ model pipeline showing embeddings generation and retrieval of matching individuals.

Section 3. We then present results on the sharkfin dataset in Section 4. We discuss the impact of various techniques to provide evidence of and detail specific success and failure cases. Finally, we overview the SHARK MATCHER tool and how its design and technical underpinnings were designed to modernize ReID workflows while addressing the needs of biological researchers in Section 5.

2 Related Work

The re-identification of individual animals is a cornerstone of wildlife conservation and ecological efforts, enabling population assessments, tracking movements, and understanding behaviors [1, 4, 15, 40]. Traditional photo identification relies on manual comparison of distinctive natural markings, such as nicks, notches, scars, and pigmentation patterns [41], a time consuming process prone to human error, especially as image databases grow. Consequently, computer-assisted and automated methods have emerged to accelerate and improve ReID accuracy.

2.1 Image-Based ReID

For many large marine species, the dorsal fin or fluke serves as the primary feature for photographic identification. Early computational approaches focused on geometric representations of the fin’s trailing edge. Finscan, proposed by Hillman et al., framed fin comparison as a string-matching problem based on edge curvature [17]. Weideman et al. introduced CurvRank [39], an integral curvature representation to improve robustness to viewpoint changes. This system was expanded to utilize fully convolutional neural networks to automatically isolate trailing edges, eliminating the need for manual edge tracing. Hughes and Burghardt introduced the first fully automated contour-based visual identification system for great white sharks from dorsal fin imagery, focusing specifically on matching unconstrained, natural images to animal identity without requiring manual intervention for fin detection and extraction [19]. However, this approach still relied heavily on hand-crafted features that do not generalize across species and are intrinsically brittle in wild environments.

More recent methods like finFindR [36] have shifted towards deep learning, using Convolutional Neural Networks (CNNs) to extract features from sub-images along the fin, making the process more general and less sensitive to image noise. Similarly, Moskvyyak et al. presented a system for robust re-identification of manta rays and humpback whale flukes using pose-invariant embeddings (PIE) [26] learned via a deep CNN and triplet loss. Finally, FIN-PRINT [7] introduced an end-to-end

automated system for killer whale recognition incorporates dedicated stages for dorsal fin and saddle patch detection, extraction, data enhancement to filter invalid images, and multi-class individual classification.

Beyond recognizing fin morphology, re-identification tasks for other species have broadened algorithm development. HotSpotter [10] is a widely used algorithm based on extracting and matching keypoints or “hotspots”. This approach emphasizes distinctive regions of an animal’s pattern and has been successfully applied to zebras, giraffes, and jaguars, among others. For sharks with unique spotting patterns, van Tienhoven et al. introduced the Interactive Individual Identification System [37]. This computer-aided program compares natural pigment markings but relies on human input to highlight key pigmentation regions, then presents matches in a ranked list. Finally, Zheng et al. proposed a transformer network structure for wild terrestrial animal ReID based on fur, stripes, and facial features, addressing limitations of traditional CNNs in capturing long-distance feature relationships [42]. These successive works highlight a growing trend towards more general, less feature-specific, fully automated deep learning architectures for both marine and terrestrial animal recognition.

Recent development has pushed this front to develop species-agnostic ReID models targeting generality and adaptability across species. Otarashvili et al. demonstrated that training deep metric learning models on large, multispecies datasets (49 species, 37K individuals) yields a generalized embedding network, MiewID [30], that outperforms single-species networks. MiewID proves that biological features learned across species create more reliable and discriminative embedding representations. By generalizing to multiple species, these frameworks address the common challenge of limited labeled training data for specific species, improving the reach of automated re-identification. The ALFRE-ID [27] pipeline offers a novel approach for species-agnostic patterned animal re-identification by combining learnable local features with fixed global understanding through feature aggregation. Local feature extractors that can be updated to address differences between species while a fixed global mode helps minimize the overhead of retraining. We combine these paradigms of leveraging generally trained ReID models with species-specific

adaptation, leveraging MiewID as a foundation model but improving its accuracy for our shark identification task through fine-tuning and updates to the match ranking method.

2.2 Multimodal Approaches

Advancements in deep learning, particularly with deep CNNs such as EfficientNet and Vision Transformers (ViTs), have produced high quality results by learning discriminative feature embeddings exclusively from images. These models perform best in ReID tasks when fine-tuned on large, accurately labeled datasets, allowing them to learn the often very subtle variations that appear across different individuals within the same species. Many wildlife datasets including the Block Lab’s sharkfin dataset used in our development include environmental metadata and field notes such as sex and length estimates of photographed individuals.

To leverage this additional data, Li et al. proposed the Meta-Feature Adapter (MFA) [23], a module that utilizes a Large Language Model (LLM) to translate environmental metadata into qualitative, natural language statements before encoding them into text embeddings. The MFA then incorporates text embeddings into image features via gated cross-attention for deep fusion. Li et al. also propose a re-ranking step to satisfy powerful geometric constraints such as time and location; instead of incorporating these constraints into the feature vectors, they are referenced after the initial ranking as symbolic constraints.

Chen et al. propose UniReID [9] to leverage metadata to address the challenge of adapting a ReID model to entirely unseen species. Unlike MetaWild, the UniReID framework does not leverage metadata from the captured image; instead, it incorporates data on the biometric properties of new species to align a general ReID model to a new species-specific task. This mechanism hones the image encoder’s focus with prompts based on representative images of a species, aligning the ReID model using semantic knowledge derived from an LLM that is queried for distinguishing features of that species.

Similarly, CoOp [46] and CLIP-ReID [22] propose mechanisms to adapt the

image encoder by generating language descriptions of specific images and classes or individuals in the dataset. Each class is therefore ambiguously described in text, expanding on generally uninformative numerical class IDs. These descriptions are then included in text-aware CLIP image encoders to support the generation of more discriminative embeddings through targeted prompting. VLM2Vec [21] demonstrates a different approach to multimodal embedding generation through the use of early or deep fusion. Rather than using separate encoders, VLM2Vec fuses visual and textual data by interleaving visual tokens with text instruction tokens and processing them jointly via self-attention, producing a single fixed-dimensional embedding vector.

Our metadata approach differs from those of CLIP-ReID and VLM2Vec in that we utilize entirely independent encoders; our image model is a specialized vision model and our metadata is encoded as a set of feature vectors. This offers greater image encoder flexibility, not requiring a VLM as the backbone model. We take heavy inspiration from the Meta-Feature Adapter to produce image and text embeddings but design representations to uniquely represent each available metadata feature instead of summarizing them with an LLM and then fusing the modalities with a cross-attention mechanism. We demonstrate how our approach produces meaningful performance improvements while allowing efficient training and offering greater feature attribution and explainability. We also develop a symbolic re-ranking approach similar in approach to that of MetaWild but leverage the Probabilistic Soft Logic (PSL) framework [3] to build and enforce constraints as templated first-order logic rules. PSL supports the construction of flexible constraints, implements weight learning, and provides efficient re-ranking based on the implemented symbolic constraints.

2.3 ReID Systems and Tools

The increasing usage and volume of photographic data has driven the development of comprehensive ReID platforms and applications, both to streamline matching and improve data accessibility and collaboration around dataset curation. Wild-

book [6] pioneered an open-source web platform that enables citizen scientists to participate in crowd-sourcing conservation through its aggregation of species detection models paired with a robust database and web-based user interface. Flukebook [8], a species specific Wildbook, offers an interface to support the identification for 15 cetacean species using a human-in-the-loop approach pairing ReID algorithms with expert verification to confirm or reject potential matches. Identifin [25] is a software built specifically for fin-based ReID with usage analysis finding that the application shortened matching time to an average of two minutes per image with 96% accuracy under optimal conditions. However, this system still showed vulnerabilities to poor image quality and environmental variance, requiring high quality photographs to function optimally. These systems illustrate the growing integration of automated identification technologies, balancing automation and expert human-in-the-loop oversight to streamline conservation biology workflows. However, interaction with these platforms revealed steep learning curves, a lack of control over the underlying model, and the need to share uploaded data. We therefore introduce an open-source ReID platform that provides an interface to our model framework with a UI co-developed with marine biologists, offering full control of model tuning and data hosting to its users.

3 Our Modeling Approach

Our goal is to train a neural model to produce discriminative embeddings that are highly sensitive to biomarkers on the dorsal fin, such as unique notches and pigmentation patterns, while remaining invariant to confounding variables such as pose, lighting, and image quality. Additionally, we leverage associated metadata such as image capture time and location or notes on attributes of the pictured shark, such as length and sex, both to expand the image embeddings and inform result ranking. Our embedding retrieval algorithm must then rank the individuals with the greatest likelihood of matching the subject of a query image. Formally, let $\mathcal{X}$ denote the space of input images, and let $\mathcal{Y} = \{y_1, y_2, \dots, y_N\}$ represent a dynamic catalog of known individual identities, each associated with a small gallery $\mathcal{G}_y \subset \mathcal{X}$ of associated images. Given a query image $x_q \in \mathcal{X}$, the objective is to identify the correct individual $y^* \in \mathcal{Y}$ such that x_q and the images in $\mathcal{G}_{y^*}$ depict the same individual. Crucially, this is formulated as a *retrieval problem*: in many real world ecological monitoring scenarios, including shark ReID, the set of known identities $\mathcal{Y}$ is dynamic: new individuals are frequently discovered and added to the gallery over time. As a result, directly training a model to predict $\mathbb{P}(y \mid x_q)$ on a closed set of labels is infeasible and does not generalize to unseen individuals encountered after training. Our modeling task is therefore decomposed into two components: (1) learning an embedding function $f_\theta : \mathcal{X} \rightarrow \mathbb{R}^d$ that maps images, optionally fused with metadata, to a representation space where those depicting the same individual lie close under a distance metric $d : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$; (2) retrieving the identities y^* that best match the query in this space.

We begin by introducing technical information about our neural foundation models (Section 3.1) and our optimizations in model training (Section 3.2). We follow with our approach to include image metadata to expand from purely an image to a multimodal approach (Section 3.3) and end with an overview of our embedding retrieval algorithms (Section 3.4).

3.1 Foundation Models

We evaluated building our image model on two different foundation backbones based on two distinct neural architectures: a Vision Transformer (ViT) and a Convolutional Neural Network (CNN). Specifically, we leverage the ViT Patch16-384 model [13] and the MiewID model [30], which is built upon an EfficientNetV2 CNN architecture [35]. The ViT model utilizes a self-attention mechanism which integrates information globally across the entire image. This core ViT innovation involves treating images as sequences of flattened 2D patches, analogous to how NLP Transformers process sequences of words or tokens. An input image is reshaped into numerous equally sized patches, which are embedded to form patch embeddings. These patch embeddings are combined with learnable position embeddings that encode spatial information. The resulting sequence of embedding vectors is then fed into a standard transformer encoder, a modular stack of alternating perceptron and self-attention blocks [38] to gain an understanding of both the features in each patch and the relationship between different patches. Our experiments use the large variant of ViT Patch16-384 with 307M parameters.

The MiewID CNN relies on localized convolutional operations to interpret and encode images. Unlike the ViT model, which processes an image globally, CNNs apply sliding filters to extract local features. As the data passes through successive convolutional and pooling layers, localized features are aggregated into increasingly complex semantic representations. The spatial makeup of the image is preserved through multidimensional feature maps. This sliding window mechanism endows CNNs with a strong localized inductive bias, whereby they inherently assume neighboring pixels are highly correlated [14], whereas ViTs learn these spatial relationships entirely from scratch through global attention. For this fine-grained image recognition task, the localized attention of CNN models appears to be highly compatible and the added flexibility of ViT’s global attention does not improve understanding. EfficientNet builds on the classic CNN architecture by uniformly scaling network depth, width, and input resolution to optimize for both accuracy and computational efficiency; it possesses a significantly smaller footprint than ViT

at approximately 54M parameters.

Beyond their architectural differences these foundation models vary significantly in their pre and post training approaches and datasets. Both the ViT and EfficientNetV2 foundation models underwent pre-training on a massive, general ImageNet dataset [11] using Cross-Entropy loss. Cross-Entropy optimizes the model to build linear decision boundaries to separate different classes. Following Cross-Entropy pretraining, these foundation models are optimized to separate highly diverse but largely non-ecological classes using linear decision boundaries. The MiewID backbone benefits from a second phase of pre-training where, to adapt EfficientNetV2 into MiewID, it is trained to learn the unique challenges of wildlife ReID. MiewID was fine-tuned on a community-curated multi-species dataset comprising 49 species, over 37,000 individuals, and 225,000 images using a sub-center ArcFace loss objective. Consequently, we observed MiewID to score significantly higher on our task, thanks to an ecological prior already trained to understand fine-grained variations among individual animals.

3.2 Fine-Tuning

To adapt both foundation models to our white shark dataset under strict hardware limits, we utilized Low-Rank Adaptation (LoRA) fine-tuning [18]. LoRA significantly improves parameter efficiency by freezing all pre-trained weights and updating only a small set of rank-decomposition matrices added to the model. This parameter efficient approach enabled model convergence within 72 hours on a single NVIDIA Quadro RTX 6000 GPU. Crucially, LoRA does not reduce the activation memory required as images pass through the model during the forward pass. A counterintuitive consequence of this, given its much smaller total parameter count (54M vs. 307M), was that fine-tuning the MiewID EfficientNetV2 backbone created a substantially higher GPU memory burden than the ViT model. Convolutional architectures must cache high resolution spatial feature maps across the model’s layers for backpropagation while the ViT model natively compresses images into smaller token sequences. Model size and high activation memory footprint severely

constrained our maximum training batch size, especially for MiewID fine-tuning. On a GPU with 24GB of memory and a training image resolution of approximately 400x400 pixels, we were limited to a maximum batch size of 8 and 32 for the MiewID and ViT models, respectively.

3.2.1 Triplet Loss

Our model aims to produce an embedding space where images of the same individual are grouped together and dissimilar images are pushed apart. We initially train using a Triplet Loss function [33] which operates on triplets of images: an “anchor” (A), a “positive” (P) (an image of the same individual as the anchor), and a “negative” (N) (an image of a different individual). The Triplet Loss is then formally defined as:

$$\mathcal{L}(A, P, N) = \max(d(A, P) - d(A, N) + \alpha, 0)$$

Here d represents the distance between any two images in the embedding space and α is a margin hyperparameter. The function’s goal is to minimize anchor-positive distance while ensuring the anchor-negative distance is larger by at least the margin α .

To train with Triplet Loss and optimize the relative distance between embeddings, we introduce a trainable Multi-Layer Perceptron (MLP) projection head atop our neural backbones. Despite the differing feature extraction mechanisms of our two foundation models, both ultimately produce a linear feature vector that is fed into the MLP. During training, the backbone and MLP are ordered sequentially; triplet optimization is done on the final outputs of the MLP. The LoRA adapted backbone and MLP are then kept in the same configuration for inference.

Model updates with Triplet Loss are greatest when training on “hard” triplets where the anchor-negative distance is initially smaller than the anchor-positive distance in the embedding space. Such hard triplets produce informative gradients for updating the model’s weights. Given the sparse nature of our dataset, where most individuals have only a few images, random batch sampling rarely yields triplets with the necessary positive pairs. This limitation reduces the effectiveness of Triplet

Loss, as many batches fail to form valid anchor-positive-negative relationships. To address this, we implemented a class-aware triplet sampler that explicitly constructs batches containing at least two positive samples for each anchor image. This approach ensures a higher proportion of valid triplets, enabling the model to learn more effectively from each training batch, notably improving retrieval accuracy.

3.2.2 ArcFace Loss

While Triplet Loss paired with class-aware sampling achieves a strong baseline, helping to improve the performance of both ViT and MiewId foundation models, it was still unable to learn from sharks with only a single image where positive samples did not exist in the dataset. Furthermore, especially during MiewID fine-tuning, GPU memory constraints limited training batch size, severely restricting the generation of informative or hard triplets due to the small sampling pool. Finally, the transition to the MiewID backbone motivated our shift from a contrastive Triplet Loss to an angular margin-based classification (ArcFace) approach. The MiewID foundation model was trained using an ArcFace Loss which transitioned the EfficientNetV2’s latent space from linear boundaries to a hyperspherical geometry [12]. Aligning our fine-tuning loss with this approach prevents objective mismatch where a break in the embedding space limits the transfer of learned knowledge. With ArcFace fine-tuning, we prevent the MiewID from reshaping its learned latent space, better preserving its priors and allowing faster fine-tuning with improved results.

Unlike Triplet Loss, which optimizes relative distances between sample pairs, ArcFace trains to form tight class clusters and clear inter-class margins on a hyperspherical embedding space. Because embeddings reside on a unit hypersphere, semantic similarity is measured by the angle between vectors, making cosine similarity the retrieval metric. Adapting model architecture for ArcFace training requires shifting from a distance based optimization to modeling a classification task. During training, the projection head becomes a classification weight matrix, where each weight vector represents the class center for a specific shark in the training set.

The ArcFace projection head acts as a classifier containing a weight matrix $W \in \mathbb{R}^{N \times D}$, where N is the number of classes and D is the feature dimension. During

training, the projection head computes cosine similarity between the L_2 -normalized image embedding and ground truth class centers. Based on their disagreement, both the embedding model weights and the class center vectors are updated. The loss is defined as:

$$\mathcal{L} = -\log \frac{e^{s(\cos(\theta_y + m_y))}}{e^{s(\cos(\theta_y + m_y))} + \sum_{j=1, j \neq y}^N e^{s \cos \theta_j}}$$

Here, θ_j represents the angle between weight W_j and feature x_i . To further increase training signal, ArcFace applies an additive angular margin penalty m to the target class angle θ_{y_i} . Logits are scaled by a factor s , to land on the hypersphere radius. We extend Arcface with a Dynamic Adaptive Margin (m_y) based on class frequency. This counteracts the long-tailed distribution of our dataset and gives individuals with very few photos a larger angular margin penalty, forcing them to be clearly separate from other classes, while individuals with numerous images are given a smaller margin. Additionally, to account for the high intra-class variance of our images, we integrated a sub-center approach where 3 sub-centers are used for each class. This allows the model to learn three different cluster centers for a single individual, a very relevant optimization for our multi-view dataset where the same individual may look significantly different depending on the image type (underwater vs topshot) or shark pose. During inference, the ArcFace classification head is dropped and the trained backbone maps representations onto the hyperspherical embedding space.

3.2.3 Data Augmentation

During fine-tuning, we observed increased adaptation to our domain but also signs of overfitting. Qualitative analysis of the embedding space revealed the model sometimes relied on spurious correlations, grouping images by irrelevant attributes such as background color or camera angle rather than individual-specific biomarkers. To mitigate this, we applied augmentations [24, 44] that altered perspective, rotation, scale, focus, and color during training. Generating two randomly augmented views per original image increased dataset diversity, improved robustness to nuisance factors, and reduced reliance on background cues. This augmentation strat-

egy ultimately decreased overfitting and improved generalization to unseen data. We selected the following augmentations following ablation experiments:

- **Contrast Limited Adaptive Histogram Equalization (CLAHE)** divides an image into tiles and calculates a histogram for each tile to enhance local contrast. To prevent increasing noise too aggressively, CLAHE clips pixel values to remain within a specific threshold across the histogram for the tile. The tiles are seamlessly combined for smooth transitions without visible boundaries between the individual tiles regions. CLAHE addresses uneven light scattering and normalizes contrast across the image to prevent biological features from being disguised by extreme color fluctuations.
- **Color Jitter** uniformly modifies the image by uniformly modifying its color channels for a global color change. This addresses water’s property to act as a light filter and cause heavy and unpredictable color changes in different depth, lighting, and water quality conditions.
- **Gaussian Blur** acts as a low pass filter that dulls high-frequency details such as sharp edges and small scale noise. This simulates low quality images with blur from dynamic subjects or turbulent water and trains the model to avoid a reliance on highly brittle features.
- **Random Adjust Sharpness** blends the original image with a sharpened version of itself to increase or decrease edge contrast, making the image appear more or less sharp. As with Gaussian Blur, this augmentation trains the model to generalize to different levels of image quality with sharper or softer region boundaries.
- **Random Erasing** selects a random rectangular region within the image and replaces its pixels with random noise or the dataset’s mean pixel value. By simulating partial occlusion, this augmentation simulates natural instances when fin regions are covered such as by bubbles, tags, or image cutoff.
- **Random Affine** geometrically transforms an image through translation and zooming while preserving the linear distance between points. This simulates variations in the subject position within the image and relative to the camera.
- **Random Perspective** performs a non-linear perspective transformation which warps the image as if viewed from a different vantage point. This augmentation

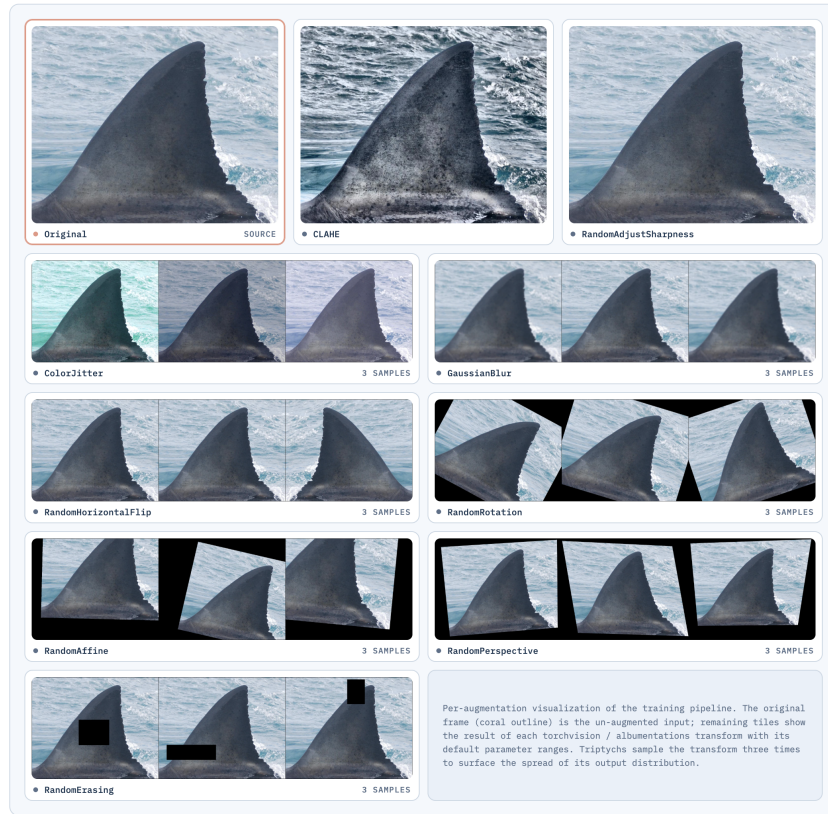


Figure 3. Visual examples showing the set of augmentations applied.

helps train for variations in camera pitch and picture angle relative to the subject.

- **Random Rotation** rotates the image around its center by a random angle within a specified range. This augmentation simulates camera roll and the angle of the subject in the water.
- **Random Horizontal Flip** mirrors the image along the vertical axis. This augmentation simulates how images can be taken from either side of the animal. Animals may possess asymmetric marks but the fin contour is symmetric.

3.3 Multimodal Approaches

In addition to images, the Block Lab sharkfin dataset contains the following metadata fields for each image: (1) image notes on quality (classified as unmatchable, poor, medium, and good); (2) a note on whether the image is underwater

or above water (topshot); (3) data on image capture time and location; (4) notes on visually observed biological attributes of the pictured shark, including length (rounded to the nearest half foot with a margin of error) and sex predictions, and ping data from acoustic and satellite tags.

We develop two techniques for including metadata features into the matching algorithm: (1) we construct MULTIMODAL EMBEDDINGS by fusing image embeddings with available metadata; (2) we apply SYMBOLIC RE-RANKING to the neural embeddings where embedding distances are updated based on metadata compatibility between the query and result. In both cases, the multimodal component builds on the existing image model; it does not require a specialized multimodal foundation model such as a Vision Language Model (VLM) or even the re-training of the image encoder model. In fact, the existing cached image embeddings are used, allowing for low overhead and rapid prototyping of the metadata representations and weights. Due to their nature as predictions and the breadth of data sources for the sharkfin dataset, the available metadata fields may contain margins of error, incorrect values, or missing values. Both multimodal approaches must be robust to these data challenges to prevent metadata noise from corrupting match suggestions and retrieval accuracy while allowing relevant features to update results.

3.3.1 Multimodal Embeddings

To build a multimodal embedding representation that combines metadata features with existing image embeddings, we apply a late fusion approach. We construct metadata representations for each feature and fuse them with the image embedding via a set of learnable weights to create multimodal representations that map to a single embedding space. The existing embedding retrieval algorithm is unmodified and can be used to fetch either image-only or multimodal embeddings without modification. We evaluate multimodal performance with both the ViT and MiewID based image models to show that with either, the addition of the metadata features provides an increase to retrieval accuracy.

Integrating heterogeneous metadata with high-dimensional image embeddings presented challenges in how to best fuse and weight individual features. Our ini-

tial approach of feature concatenation used a trainable projection head to fuse data. In this implementation, both image and metadata features are transformed by separate MLPs to be projected to a common shared dimension. Then the two projected embeddings are concatenated and passed through a single layer projection head, trained to learn cross-modal interactions. This approach struggled with the dimensionality mismatch and frequently sparse or missing metadata fields which led to overfitting on the training set and reduced retrieval accuracy compared to the image-only baseline. To address this, we transitioned to a weighted late fusion approach combined with a metadata encoding strategy specialized for each feature’s data properties.

Our fusion strategy produces an interpretable, weighted sum of independent modalities. Let G represent the set of distinct metadata feature groups (e.g., sex, location, time), and let x_{img} denote the frozen image embedding. We independently L_2 -normalize the image embedding into $\hat{x}_{img}$ and each engineered metadata feature vector into $\hat{e}_g$. Each normalized vector is then scaled by a tuned scalar weight α , concatenated, and re-normalized to produce the final fused embedding f :

$$f = L_2norm([\alpha_{image} \cdot \hat{x}_{img}, \alpha_{g_1} \cdot \hat{e}_{g_1}, \dots, \alpha_{g_n} \cdot \hat{e}_{g_n}])$$

This late fusion process ensures that the final embedding is governed purely by the relative magnitudes of the tuned α weights. Because the fusion mechanism does not rely on a complex trained head, we prevent the image signal from being modified or drowned out by the metadata, allowing the model to naturally fall back to the unaltered visual embeddings when metadata is sparse or uninformative.

As with image embeddings, metadata vectors must mirror the similarity of the features they represent. Simple continuous values or one-hot vectors with an *unknown* class fail to accurately mirror how features support or reduce semantic similarity. They may, for example, falsely reward two samples for both lacking data or equally penalize two samples with missing metadata and two samples with directly conflicting metadata. Because our fused multimodal representations are L_2 -normalized to reside on the same unit hypersphere as the image embeddings, our retrieval metric remains cosine similarity. We therefore design our metadata

representations to map semantic matches with positive cosine values, conflicts with negative values, and missing data with a neutral signal through a cosine similarity of 0.

For categorical fields such as sex and location, non-missing data is one-hot encoded and then centered by the population class proportions before L_2 -normalization. Continuous features such as shark length are encoded using a binned Radial Basis Function (RBF) mapping. This creates a distance graded similarity, where similar values yield a cosine similarity near 1, and distant values approach 0 to reflect the acceptable margin of error in shark length prediction. Missing values are imputed using the population mean to provide a probable loss instead of producing an outlier signal. Cyclical time-of-year values are represented by sine projection to provide consistent behavior across the calendar year boundary. Each feature group is independently L_2 -normalized to decouple scale differences from group-importance weights α . The concatenated vector is then L_2 -normalized to place all fused embeddings on the unit hypersphere, enabling retrieval by the existing cosine similarity retrieval function.

We optimize the α weighting parameters for the image embedding and each metadata feature through a hyperparameter tuning search. By operating strictly on precomputed per-group similarity matrices rather than performing raw image forward passes, we can quickly evaluate thousands of trial weightings utilizing a combination of random search and coordinate refinement within minutes. A benefit of this technique is the ability to directly optimize for our retrieval metric with our objective function targeting lowest average K ranking and validating across multiple data splits held strictly separate from the test set.

3.3.2 Symbolic Re-Ranking

SYMBOLIC RE-RANKING is a post-hoc approach that applies metadata constraints into the ranked neighbor lists produced by embedding retrieval. We formulate these constraints as soft logical rules within the Probabilistic Soft Logic (PSL) framework [3]. PSL represents logical predicates as continuous or soft truth values in $[0, 1]$. This allows explicitly expressing physical and biological constraints

as templated rules. If the neural model falsely flags a high visual similarity between two sharks separated by impossible constraints, the PSL framework will enforce a metadata rule to penalize that match and push it down the ranking. Inference aims to find the joint assignment of truth values for all unobserved predicates that minimizes a weighted sum of soft constraint violations. For a given query, this re-ranks the top- K neural neighbors such that biologically conflicting candidates are penalized and consistent ones are promoted.

Our PSL model is defined over five predicates which reflect discriminative qualities that inform shark matching. The observed predicate $\text{NeuralSim}(I_1, I_2)$ encodes the pairwise neural similarity between images; this acts as the model’s prior. Two observed predicates $\text{IsMale}(I)$ and $\text{IsFemale}(I)$ encode the sex feature. The pairwise observed predicate $\text{SimilarLength}(I_1, I_2)$ encodes soft length compatibility. The target predicate, $\text{SameShark}(I_1, I_2)$, is the continuous probability of a match which is optimized for across all query-neighbor pairs during inference.

Given these predicates, we apply five weighted rules constrain the inference:

$$w_p : \text{NeuralSim}(I_1, I_2) \rightarrow \text{SameShark}(I_1, I_2) \quad (1)$$

$$w_p : \text{SameShark}(I_1, I_2) \rightarrow \text{NeuralSim}(I_1, I_2) \quad (2)$$

$$w_s : \text{IsMale}(I_1) \wedge \text{IsFemale}(I_2) \rightarrow \neg \text{SameShark}(I_1, I_2) \quad (3)$$

$$w_s : \text{IsFemale}(I_1) \wedge \text{IsMale}(I_2) \rightarrow \neg \text{SameShark}(I_1, I_2) \quad (4)$$

$$w_l : \neg \text{SimilarLength}(I_1, I_2) \rightarrow \neg \text{SameShark}(I_1, I_2) \quad (5)$$

The two-sided neural prior (Rules 1–2) anchors SameShark to the neural ranking. The forward rule lifts match probability in proportion to neural similarity, while the reverse rule prevents saturation. Without this reverse, inference can push all candidates toward the prior maximum regardless of biological evidence, collapsing the range of probability scores. The sex disagreement rules (Rules 3–4) exploit the biological certainty that individual sharks have a fixed sex while incorporating robustness to labeling errors through flexible weighting. The length disagreement rule (Rule 5) penalizes pairs whose observed lengths differ beyond what the 3 foot margin of error our system accounts for.

As with MULTIMODAL EMBEDDINGS, a prerequisite of this approach is that it remain silent when metadata is missing. Under PSL’s Łukasiewicz t-norm semantics, a rule body evaluates to $\max(0, \sum_i \text{atom}_i - (n - 1))$ for n body atoms, which reaches zero whenever any atom is zero, producing no penalty. Sex is represented as a hard binary pair: known male as (1.0, 0.0) and known female as (0.0, 1.0), but unknown sex as (0.0, 0.0). Encoding unknown sex as hard zeros drives the body to zero regardless of the other image’s sex value, keeping the rules completely silent on unknown values. Length differences within the accepted margin of error of 3 feet yield `SimilarLength` = 1.0, making the rule body $\neg \text{SimilarLength} = 0$, keeping the constraint silent. Differences of 3 to 6 feet are linearly interpolated from 1.0 to 0.0, and differences of 6 feet or more are penalized most heavily with `SimilarLength` = 0.0, fully activating the length constraint rule. The rule therefore fires only when both lengths are known and their difference exceeds the error margin.

Rule weights are learned to reflect each constraint’s relationship with the true labels using the PSL framework’s maximum-likelihood weight learner. We assemble a single Hinge-Loss Markov Random Field (HL-MRF) over the top- $K = 100$ neighbor lists of all training queries, supervised with ground-truth `SameShark` labels derived from individual identity (1 for a true match, 0 otherwise). The learner alternates inference under the current weights with a gradient step that compares each rule’s distance-to-satisfaction in the inferred state against the labeled state. Weights are raised when a rule is more violated under the model than under the ground truth and lowered otherwise, until convergence. The neural prior weight is fixed and excluded from optimization. This is because, by definition, most top- K candidates are incorrect matches; with high `NeuralSim`, Maximum Likelihood Estimation (MLE) over the training set collapses w_p toward zero to minimize aggregate training energy, which destroys the neural ordering that re-ranking depends on.

At inference, the top- K neural neighbors of a query are converted from their embedding distances to PSL truth values. For L_2 -normalized embeddings, the cosine similarity is $\cos \text{sim}(I_1, I_2) = 1 - d^2/2$, where d is the embedding distance, and

the PSL truth value is $\text{NeuralSim} = \max(0, (\cos \text{sim} + 1)/2)$, mapping cosine similarity from $[-1, 1]$ to the $[0, 1]$ PSL truth range. The top- $K = 100$ candidates are re-sorted by descending inferred match probability. We skip PSL learning and re-ranking beyond $K = 100$ for computational efficiency; these results maintain their original neural order with distances simply projected to PSL’s $[0, 1]$ distance space.

3.4 Retrieval Algorithms

Once the embedding function f_θ has been trained, the entire dataset of images, optionally with metadata included, are encoded to a shared representation space. Given a query image, retrieval is performed by ranking candidate identities according to their similarity to the query in this space. A well-trained embedding model should organize images into distinct, well-separated clusters, each corresponding to a unique individual.

We evaluate two retrieval strategies: (1) NEAREST NEIGHBOR (NN) retrieval compares the query embedding directly to all catalog image embeddings and ranks identities based on the closest matches; (2) NEAREST PROTOTYPE (NP) retrieval first computes a representative prototype embedding for each individual by averaging all its associated image embeddings, then compares query proximity to these prototypes [32]. Prototype based retrieval reduces the search space from the total number of images to the number of prototypes, equal to the number of unique identities in the dataset. This approach is more robust to intra-individual variation producing inaccurate matches through factors independent of shark identity. Formally, let the embedding of a query image x_q be $z_q = f_\theta(x_q)$, and let the embedding set for identity y be $Z_y = \{f_\theta(x) \mid x \in \mathcal{G}_y\}$. Retrieval ranks candidates using a distance metric $d : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$:

Nearest Neighbor (NN) Retrieval. Rank all identities by their closest image embedding:

$$\hat{\mathcal{Y}}_{\text{NN}} = \text{rank} \left(y \in \mathcal{Y} \mid \min_{z \in Z_y} d(z_q, z) \right)$$

Nearest Prototype (NP) Retrieval. First compute the prototype for each identity:

$$\mu_y = \frac{1}{|Z_y|} \sum_{z \in Z_y} z,$$

then rank identities by their prototype distance:

$$\hat{\mathcal{Y}}_{\text{NP}} = \text{rank} \left(y \in \mathcal{Y} \mid d(z_q, \mu_y) \right).$$

By averaging over all available images for an individual, NP retrieval smooths over nuisance variation and background noise, often improving accuracy in dynamic catalog settings. A visual example is presented in Figure 4.

Following retrieval, we evaluated applying a re-ranking step based on k-reciprocal encoding [43]. This method relies on mutual agreement where two images are only considered a strong match if they appear in each other’s top- k nearest neighbors. Formally, the k-reciprocal set $\mathcal{R}(i, k)$ of an image i is the subset of its k nearest neighbors j such that i itself appears among j ’s k nearest neighbors. To capture transitive neighbor relationships, this set is then expanded to include the half-size ($k/2$) reciprocal sets of its neighbors if those sets overlap by at least a margin of $2/3$. Each image is then encoded as a sparse feature vector V_i , where an entry is zero if not part of the reciprocal neighborhood and otherwise non-zero. Non-zero are weighted by a Gaussian decay based on their original distance and subsequently L_1 -normalized. To measure the full structural similarity between a query and a matching image, the Jaccard distance between their respective reciprocal feature vectors is computed. This quantifies the overlap of their local neighborhoods. This Jaccard distance is then blended with the original embedding distance, with the relative weighting of the two vectors defined by a value λ , to produce the final distance metric. During testing, we found that for NP retrieval, k-reciprocal encoding’s distance calculation consistently improves retrieval results, we therefore apply this technique in all cases of NP retrieval. In NN retrieval, where the gallery is the full set of images and the neighborhood structure is noisier, k-reciprocal encoding produces no comparable benefit and often worsens accuracy so we exclude the technique from this retrieval algorithm.

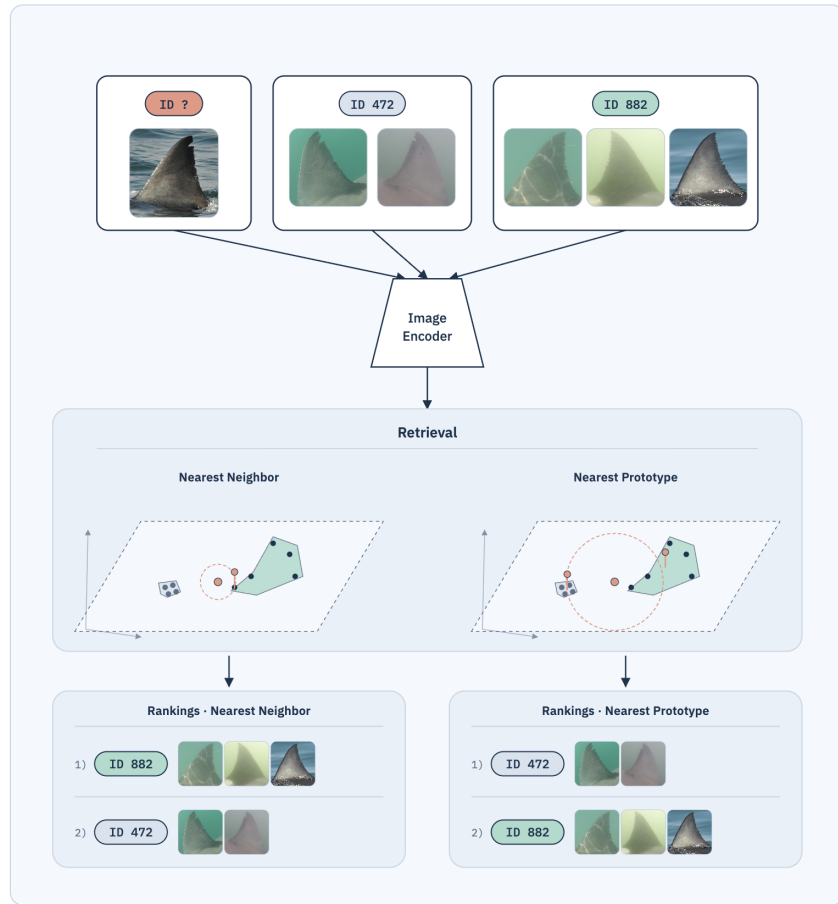


Figure 4. A visual comparison between NN vs. NP retrieval. Each image maps to a point in embedding space. NN retrieval compares image embeddings while NP retrieval builds a single prototype for each individual.

4 Empirical Evaluation

Our evaluations aim to quantify the individual contributions each of our model approaches has to retrieval accuracy. After introducing our dataset and targeted evaluation metrics in Section 4.1, we begin by evaluating image-only embeddings without the addition of metadata in Section 4.2. Here, we compare the performance of the ViT and MiewID foundation backbones and the impact of different fine-tuning techniques. We continue by comparing image-only results with multimodal ReID in Section 4.3, showing the benefit both MULTIMODAL EMBEDDINGS and SYMBOLIC RE-RANKING have on key ReID metrics. We follow this with a comparison of nearest neighbor and nearest prototype retrieval in Section 4.4. We then drill down on results in Section 4.5 to show notable performance differences when evaluating specific subsets of the dataset, including images of higher or lower quality or rarely seen sharks with few associated images. We also compare the results of models trained on two separate snapshots of the sharkfin dataset to quantify how improved training data quality through labeling corrections improves outcomes.

4.1 Dataset

We evaluate on a dataset of dorsal fin images curated by the Block Lab at Stanford University [2]. This sharkfin dataset contains images captured between 1994 and 2026, with many individuals re-sighted and photographed years apart. As of April 2026, the dataset contained 4193 images spanning 1376 individual sharks. The data also exhibits a long tail distribution, common in wildlife datasets, with 680 sharks containing only a single associated image. 696 sharks contain two or more images, 230 contain 5 or more images, and 16 sharks have been photographed and matched 20 or more times. Temporal and shark identity distributions, shown in Figure 5, show the identity distribution and steady growth of the image dataset over the years. Critically, our data set has been curated to contain only a single image of any individual shark encounter; there are no duplicate or burst images with trivial differences in capture time, which would artificially increase average retrieval scores.

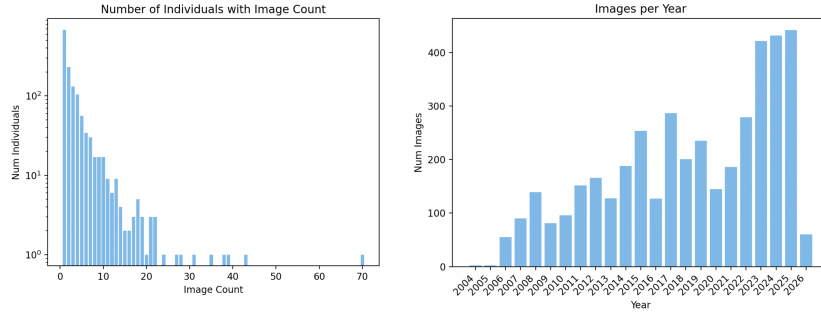


Figure 5. Number of sharks with counts of matched images showing a long-tailed class distribution (left). Number of images by year (right).

We use two different methods to create held-out test sets, which we refer to as IMAGE REMOVAL and SHARK REMOVAL. IMAGE REMOVAL creates a test set by randomly selecting one image per shark to be included in the test set for each shark with two or more associated images. Thus, all sharks represented by test images were encountered during model training. In contrast, SHARK REMOVAL builds a test set out of only the newest sharks, using all of their images. Thus, none of the images for any test sharks are used in training. The two approaches, summarized in Table 1 and Figure 6, highlight different objectives. IMAGE REMOVAL evaluates how well the model matches existing sharks while SHARK REMOVAL evaluates how well the model generalizes to new, unseen sharks.

	Image Removal	Shark Removal
Image Count	696	842
Shark Count	696	251
Sharks with 2 Images	230	236
Sharks with 3-5 Images	292	394
Sharks with 6+ Images	174	212
Unknown Quality Images	270	12
Unmatchable/Poor Quality Images	52	105
Medium/Good Quality Images	374	725

Table 1. Comparison of the IMAGE REMOVAL and SHARK REMOVAL test sets. The two approaches show a similar representation associated image count and a quality distribution.

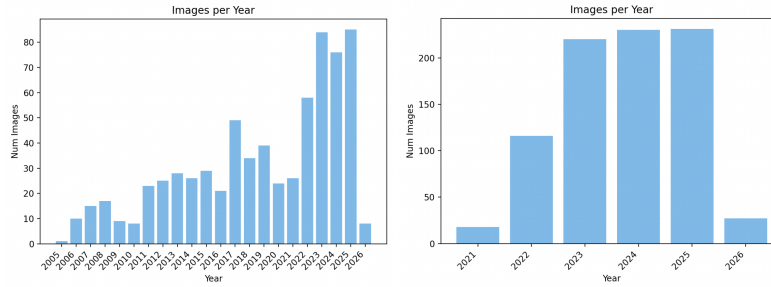


Figure 6. Number of images by year in IMAGE REMOVAL selection (left) and SHARK REMOVAL (right); IMAGE REMOVAL evaluates sharks spanning the full temporal range of the dataset while SHARK REMOVAL evaluates performance on newer images.

We benchmark performance using Hits@ K , which measures the proportion of queries for which the correct individual is present within the top K retrieved individuals. We report scores up to $K = 25$, representing a practical number of match suggestions for human review. We also evaluate the Area Under the Cumulative Match Characteristic Curve (AUC-CMC). The Cumulative Match Characteristic (CMC) curve plots the probability of finding the correct identity within the top K ranks, effectively plotting Hits@ K for all possible values of K . The AUC-CMC is the area under this curve, normalized to a value between 0 and 1. AUC-CMC therefore summarizes the full spectrum of Hits@ K results in a single metric where a high value indicates a consistently high ranking of the correct shark. While Hits@ K quantifies retrieving the first correct match, Mean Average Precision (mAP) measures the ability to retrieve all matching images of a queried individual. mAP therefore provides a comprehensive view of how well the model ranks all images of a matching individual near the top of the search results, rather than just locating a single good match.

4.2 Foundation Models and Fine-Tuning

We begin with a comparison of image model performance to showcase the accuracy baselines of the ViT and MiewID foundation models and the impact of fine-tuning with triplet and arcface loss functions in Table 2 and Table 3. The MiewID

foundation model, thanks to its second stage of pre-training on a large ReID specific multispecies dataset, begins at a far higher baseline. This shows the significant benefit of task specific pre-training on an extensive wildlife dataset with the MiewID baseline outperforming our best performing ViT based model. Despite its high starting accuracy, we measure significant improvement through fine-tuning MiewID, especially after switching from triplet loss to arcface loss, which is better aligned to MiewID’s pretrained embedding space.

The trends of model and fine-tuning performance are mirrored in both the IMAGE REMOVAL and SHARK REMOVAL selections. Results are marginally better across the board over the IMAGE REMOVAL selection. We hypothesize this is due to each individual shark having been trained on with this approach.

Backbone + Fine-Tuning	Image Removal Results				
	Hits@1	Hits@5	Hits@25	AUC-CMC	mAP
ViT Baseline	0.03	0.09	0.16	0.23	0.03
ViT Triplet	0.51	0.72	0.86	0.88	0.50
MiewID Baseline	0.83	0.91	0.94	0.95	0.77
MiewID Triplet	0.84	0.92	0.95	0.96	0.79
MiewID ArcFace	0.89	0.94	0.96	0.97	0.87

Table 2. IMAGE REMOVAL retrieval accuracy comparison for baseline and fine-tuned image models by loss function (Triplet vs. ArcFace).

Backbone + Fine-Tuning	Shark Removal Results				
	Hits@1	Hits@5	Hits@25	AUC-CMC	mAP
ViT Baseline	0.03	0.08	0.16	0.22	0.03
ViT Triplet	0.51	0.75	0.89	0.90	0.43
MiewID Baseline	0.83	0.91	0.94	0.95	0.77
MiewID Triplet	0.83	0.91	0.95	0.96	0.77
MiewID ArcFace	0.89	0.94	0.96	0.96	0.84

Table 3. SHARK REMOVAL retrieval accuracy comparison for baseline and fine-tuned image models by loss function (Triplet vs. ArcFace).

4.3 Multimodal Approaches

We compare image-only and multimodal results in Table 4 and Table 5. Using both the best performing fine-tuned ViT and MiewID based models, we show the impact of including metadata information through our MULTIMODAL EMBEDDINGS and SYMBOLIC RE-RANKING techniques. Results show that regardless of the underlying image model, both MULTIMODAL EMBEDDINGS and SYMBOLIC RE-RANKING increase performance across all metrics we track. These approaches are not mutually exclusive, with the combination of both most often achieving the best performance. Additionally, we observe a slightly higher boost in performance with the SYMBOLIC RE-RANKING technique on the SHARK REMOVAL results, curbing the trend of generally lower accuracy on this selection. We hypothesize this is due to the more frequently available metadata in the newer test images present in SHARK REMOVAL, allowing multimodal approaches to have their greatest possible impact.

Model	Image Removal Results				
	Hits@1	Hits@5	Hits@25	AUC-CMC	mAP
ViT Image	0.51	0.72	0.86	0.88	0.50
ViT + ME	0.58	0.76	0.88	0.90	0.55
ViT + SR	0.56	0.76	0.88	0.89	0.54
ViT + ME + SR	0.61	0.78	0.90	0.91	0.58
MiewID Image	0.89	0.94	0.96	0.97	0.87
MiewID + ME	0.90	0.95	0.98	0.98	0.88
MiewID + SR	0.87	0.94	0.97	0.97	0.88
MiewID + ME + SR	0.91	0.96	0.98	0.98	0.89

Table 4. IMAGE REMOVAL retrieval accuracy comparison for image-only and multimodal approaches for best performing ViT and MiewID models. ME is MULTIMODAL EMBEDDINGS. SR is SYMBOLIC RE-RANKING.

Model	Shark Removal Results				
	Hits@1	Hits@5	Hits@25	AUC-CMC	mAP
ViT Image	0.51	0.75	0.89	0.90	0.43
ViT + ME	0.58	0.80	0.90	0.92	0.49
ViT + SR	0.55	0.80	0.91	0.92	0.48
ViT + ME + SR	0.60	0.81	0.91	0.92	0.50
MiewID Image	0.89	0.94	0.96	0.96	0.84
MiewID + ME	0.90	0.95	0.97	0.98	0.85
MiewID + SR	0.91	0.95	0.96	0.96	0.86
MiewID + ME + SR	0.90	0.95	0.98	0.98	0.86

Table 5. SHARK REMOVAL retrieval accuracy comparison for image-only and multimodal approaches for best performing ViT and MiewID models. ME is MULTIMODAL EMBEDDINGS. SR is SYMBOLIC RE-RANKING.

4.4 Retrieval Algorithms

Comparison between NN and NP retrieval techniques in Table 6 and Table 7 shows that the latter outperforms standard nearest neighbor retrieval, regardless of the underlying embedding model or the use of image-only or multimodal modeling. By aggregating feature representations into a single prototype for each shark, NP retrieval achieves more stable shark representations, minimizing the impact of noise and resulting in a higher accuracy ranking in retrieval. During evaluation, query images were always held out of prototype computation to simulate the setting where the query’s label is truly unknown. It is also worth noting that we have not combined SYMBOLIC RE-RANKING with NP retrieval since re-ranking is done on a per-image level.

Results reveal that models across the retrieval method comparison, including lower performing embedding models such as ViT based models, see meaningful improvements in hits across all K values and mAP. In fact, as with multimodal approaches, a significant positive impact of optimizing the retrieval technique is on mAP results. Stacked with higher performing image models and multimodal approaches, NP retrieval continues to reliably boost accuracy. Further analysis reveals that NP retrieval significantly reduces the number of queries where the model

Model	Nearest Neighbor Retrieval			Prototype Retrieval		
	Hits@1	Hits@5	mAP	Hits@1	Hits@5	mAP
ViT Image	0.51	0.72	0.50	0.59	0.77	0.65
ViT + ME	0.58	0.76	0.55	0.60	0.81	0.66
MiewID Image	0.89	0.94	0.87	0.91	0.96	0.93
MiewID + ME	0.90	0.95	0.88	0.92	0.97	0.94

Table 6. IMAGE REMOVAL accuracy comparison for NN and NP retrieval with best performing ViT and MiewID models.

Model	Nearest Neighbor Retrieval			Prototype Retrieval		
	Hits@1	Hits@5	mAP	Hits@1	Hits@5	mAP
ViT Image	0.51	0.75	0.43	0.58	0.81	0.66
ViT + ME	0.59	0.80	0.59	0.64	0.82	0.70
MiewID Image	0.89	0.94	0.84	0.90	0.96	0.92
MiewID + ME	0.90	0.95	0.85	0.90	0.96	0.92

Table 7. SHARK REMOVAL accuracy comparison for NN and NP retrieval with best performing ViT and MiewID models.

performs very poorly. Table 8 shows that the number of queries where the correct match lies at a rank of 50 or greater. These values drop most significantly through the addition of MULTIMODAL EMBEDDINGS and the use of NP retrieval. These results highlight that, for fine-grained retrieval tasks, how the embedding space is searched can be as critical as how its embeddings are constructed.

4.5 Data Quality

Closer inspection of retrieval results reveals a strong positive correlation between retrieval performance and the number of images available of a specific individual. The consistency of this trend is particularly meaningful in the context of NP retrieval, where all individuals, regardless of image count, have exactly one prototype embedding. NP’s single embedding per shark verifies that retrieval of the correct identity is more reliable not because of a higher number of matching

Model/Retrieval	Hits Below 50	Hits Above 50
Image-only, NN Retrieval	1488	50
Image-only, NP Retrieval	1515	23
Multimodal, NN Retrieval	1513	25
Multimodal, NP Retrieval	1528	10

Table 8. Number of queries across the IMAGE REMOVAL and SHARK REMOVAL test sets where the correct match lies within the first 50 results.

image embeddings in the search space, as could be the case with NN retrieval. Instead, it is evidence of more robust prototypes when more data points per shark are available. Similarly, retrieval performance increases significantly as query image quality improves. We quantify performance differences across our dataset by presenting bucketed results for both NN and NP retrieval using our strongest performing embedding model in Table 9 and Table 10.

	Nearest Neighbor Retrieval			Prototype Retrieval		
	Hits@1	Hits@5	mAP	Hits@1	Hits@5	mAP
2 Images/Shark	0.80	0.89	0.84	0.81	0.91	0.84
3-5 Images/Shark	0.94	0.99	0.90	0.94	0.99	0.96
6+ Images/Shark	0.97	0.98	0.91	0.93	0.97	0.96
Unmatchable/Poor	0.62	0.79	0.61	0.67	0.86	0.74
Medium/Good	0.93	0.97	0.90	0.94	0.98	0.95

Table 9. IMAGE REMOVAL retrieval accuracy of the fine-tuned MiewID image-only model, bucketing results by query shark frequency and image quality.

To better understand the impact of noisy data including duplicate sharks, where multiple shark classes track a single real-world shark, and mismatched images, where images are matched to an incorrect shark, we include evaluation on two distinct snapshots of the sharkfin dataset. Our most up-to-date data snapshot, the FULL snapshot, was used to produce all prior results. This snapshot reflects the dataset through April 2026 and benefits from numerous iterations of de-duplication, auditing of mismatched images, and the largest number of total images with 4193. We

	Nearest Neighbor Retrieval			Prototype Retrieval		
	Hits@1	Hits@5	mAP	Hits@1	Hits@5	mAP
2 Images/Shark	0.77	0.87	0.81	0.86	0.94	0.88
3-5 Images/Shark	0.94	0.98	0.90	0.96	0.99	0.97
6+ Images/Shark	0.94	0.97	0.80	0.95	0.98	0.96
Unmatchable/Poor	0.66	0.83	0.62	0.69	0.89	0.75
Medium/Good	0.93	0.96	0.88	0.94	0.98	0.95

Table 10. SHARK REMOVAL retrieval accuracy of the fine-tuned MiewID image-only model, bucketing results by query shark frequency and image quality.

compare the FULL snapshot with an INITIAL snapshot from April 2024, before data cleanup using Shark Matcher began. Table 11 highlights both additional data and the extent of labeling corrections made between the INITIAL and FULL snapshots. With no modifications to foundation model architecture, fine-tuning, multimodal approaches, or retrieval, we observe higher performance across retrieval metrics with the higher quality FULL snapshot. These results demonstrate the flywheel effect of leveraging the model framework to audit a dataset, retrain, and see improved ReID performance downstream. Together with the Block Lab, we repeated multiple rounds of this iterative cycle and were able to produce both a cleaner dataset and stronger model.

	Initial Snapshot	Full Snapshot
Image Count	2826	4193
Shark Count	1031	1376
Unchanged Sharks	-	650
Duplicate Shark Merges	-	361
Mismatch Corrections	-	20

Table 11. INITIAL and FULL snapshot comparison showing the amount of additional data added and extent of labeling updates between the two.

Model	Nearest Neighbor Retrieval			Prototype Retrieval		
	Hits@1	Hits@5	mAP	Hits@1	Hits@5	mAP
Initial MiewID Image	0.88	0.93	0.84	0.90	0.96	0.92
Initial MiewID + ME + MR	0.90	0.96	0.87	0.91	0.97	0.93
Full MiewID Image	0.89	0.94	0.87	0.91	0.96	0.93
Full MiewID + ME + MR	0.91	0.96	0.89	0.92	0.97	0.94

Table 12. IMAGE REMOVAL retrieval accuracy comparison for image-only and multimodal models trained on the INITIAL and FULL dataset snapshots.

Model	Nearest Neighbor Retrieval			Prototype Retrieval		
	Hits@1	Hits@5	mAP	Hits@1	Hits@5	mAP
Initial MiewID Image	0.88	0.94	0.84	0.91	0.97	0.93
Initial MiewID + ME + MR	0.91	0.95	0.86	0.91	0.96	0.92
Full MiewID Image	0.89	0.94	0.84	0.90	0.96	0.92
Full MiewID + ME + MR	0.90	0.95	0.85	0.90	0.96	0.92

Table 13. SHARK REMOVAL retrieval accuracy comparison for image-only and multimodal models trained on the INITIAL and FULL dataset snapshots.

5 Shark Matcher

SHARKLINQ’S impact on applied ecological tasks relies on having both a well performing ReID model and integrating it with a tool for biological catalog management. To address the need for a user interface to support their full workflow needs, we collaborated with the Block Lab and developed SHARK MATCHER in parallel to model and retrieval research. We built a human-in-the-loop ReID and database management tool that also allows users to view and edit shark and image properties within a single system. Additional focus was given to allow SHARK MATCHER to support multi-user access with tiered account types for greater collaboration.

Before SHARK MATCHER, the Block Lab’s marine researchers relied on highly manual, disjointed workflows. Image data was stored in standard operating system directories while the corresponding metadata and matching details were stored in spreadsheets. Viewing the shark catalog to explore both the image gallery and metadata was not possible without accessing multiple sources. This fragmented layout was a bottleneck for data intake and ReID: researchers were limited not only by having to manually find and verify matching shark identities but also by difficulty of viewing and navigating the database. Due to these difficulties and data often being available only on the single computer it was stored, the Block Lab did much of their matching and labeling with printed out photos they sorted into piles on a large table.

These challenges made it infeasible for researchers to match new images within the same day or even week they were captured. Images were often matched only within single day partitions or against only a subset of the catalog, for example matching within only a single season. These multiple partitions resulted in a heavily fragmented dataset that would need to be merged to achieve a fully matched catalog. Researchers would convene for hours in end-of-season matching sessions to manually resolve identities across partitions. This legacy approach delayed the availability of fully labeled data and introduced labeling errors due to the difficulty of tracking fragmentation across partitions.

5.1 Architecture

SHARK MATCHER is built as a full-stack application pairing a Sqlite database with a Python-based backend and API and a web UI. The tool interacts with the model framework for training, embedding generation, and embedding retrieval. SHARKLINQ comprises two adjacent software packages, summarized in Figure 5, where SHARK MATCHER (*shark-matcher*) houses the data ingestion interface, database, server, and frontend code while SHARKLINQ MODEL (*sharklinq-model*) contains code supporting model training, inference, and all matching and prediction logic. This dual repository structure allows the entire software to be distributed as a dual-container application with a lightweight server container and a separate model container for more intensive training, embedding generation, and retrieval.

To run SHARK MATCHER, users install both the *shark-matcher* and *sharklinq-model* repositories, available on GitHub, onto a machine they intend to use as the server. To fine-tune a model using the framework, *shark-matcher* can be cloned onto a GPU enabled device for training, with the trained models transferred to the server for embedding generation and inference. Data for both the *shark-matcher* and *shark-matcher-app* directories is stored in a common *sl-data* directory with a clearly defined directory structure for images and relational data. Documentation and examples for formatting relational data CSV files and an example dataset are provided. This same CSV formatted data is also utilized by the model framework, which includes scripts for data validation, generating data splits, model training, and evaluation.

5.1.1 Object Types

SHARK MATCHER supports searching the database, uploading new shark images, updating data points about individual sharks and images, and matching images to sharks. The core of SHARK MATCHER is the ReID workflow where images are labeled by the identity of the pictured shark with the SHARKLINQ MODEL recommending a ranked list of suggested matches for a queried image. SHARK MATCHER relies on a few clearly defined object types and their interactions to build its func-

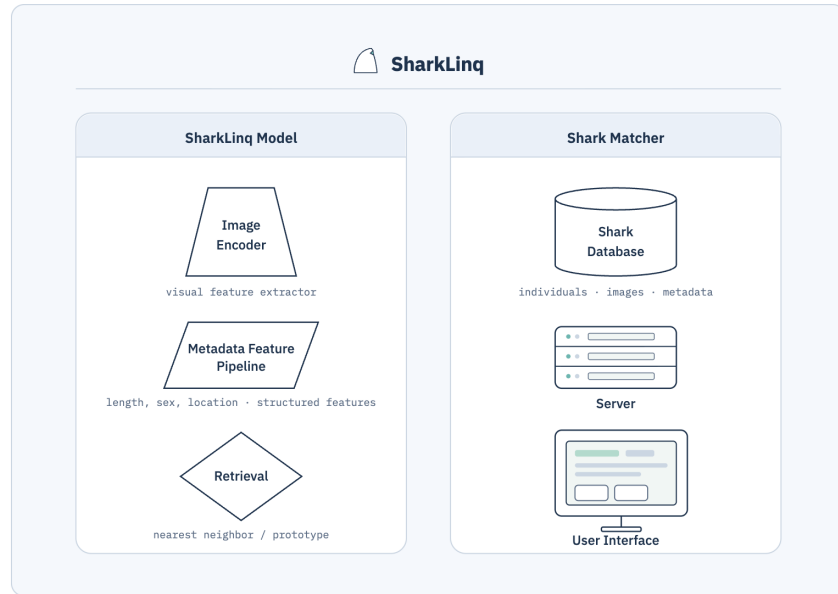


Figure 7. Overview of SHARKLINQ’s dual repository structure.

tionality. The primary object type in SHARK MATCHER is the *shark entry*. A shark entry corresponds to a real world shark. Each shark entry has a collection of associated images, the set of images mapped to that shark. Shark entries also store properties such as the sex and length of the shark they represent along with collections of associated tags, labels, and nicknames.

SHARK MATCHER stores images and their properties, including capture date, location, and a quality score with the *image* object. Additional information stored with images include notes about the pictured shark, such as sex and length estimates. Crucially, the image-specific properties do not override the properties for the shark as a whole; instead, they allow for finer grained documentation of the shark. For example, over multiple years, images of a shark may show the shark growing in length. Each image begins by being associated with a shark entry containing just that single image. Matching images can then be merged to mirror that they both refer to the same underlying animal. When an image is unmatched from a shark entry, it once again becomes associated with a freestanding shark entry that can be merged again.

Nicknames, labels, and tags are three types of objects that can be attached to

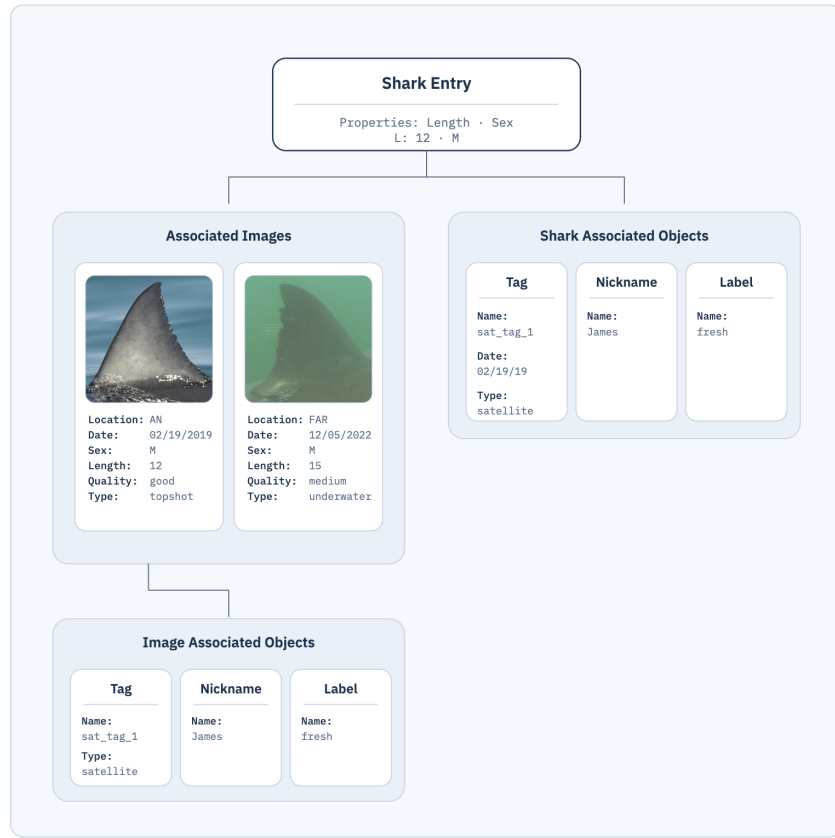


Figure 8. Overview of the SHARK MATCHER data architecture, showing the relationship between shark entries, associated images, and associated objects.

shark entries or specific images to store additional information. These three object types are collectively called *associated objects*. Nicknames allow users to give a shark or image a nickname or alias to make it easier to search and identify. Labels can be assigned to groups of sharks or images that fall into a certain category. An example of a label is the “fresh” label, applied by default to all new shark entries created when images are uploaded. Tags represent physical tracking tags that can be deployed on animals. The storage of tags in SHARK MATCHER also allows researchers to view and manage image matching and tag deployment data within one system. Associated objects may also aid matching by providing additional context for human verifiers. While they are not currently included in the MULTIMODAL EMBEDDINGS approach or weighted by the retrieval algorithm, these are both potential

areas for future work.

5.1.2 Workflows

Shark Management Each shark entry has a profile page (Figure 9) where users can view and edit the shark’s properties, associated objects, and associated images. The shark’s associated images are visible in the image gallery towards the bottom of the page. The leftmost image in the gallery is the shark’s reference image which serves as the profile picture for that shark entry; it always appears first in the image gallery and can be set to any of an entry’s associated images. Selecting any of a shark’s associated images allows for editing that image. Users can update image metadata, unmatched the image from the shark entry, or delete the image from the database entirely.

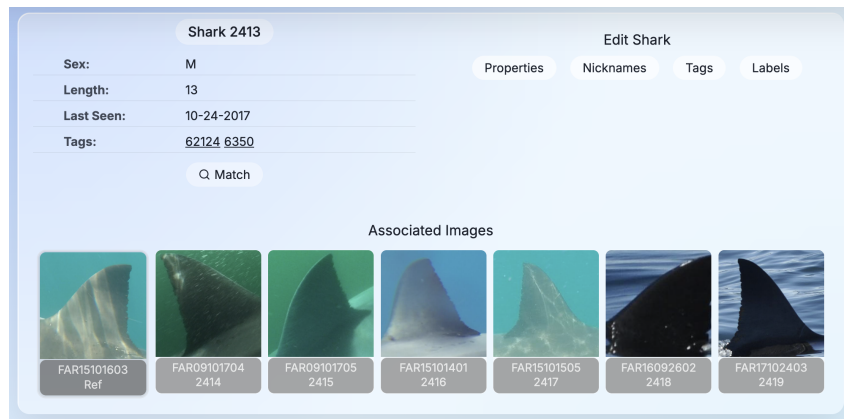


Figure 9. The shark profile page displays a shark entry’s properties, associated images, and associated objects.

Upload The entry point to SHARK MATCHER is data upload and ingestion (Figure 10). Users can upload newly captured images directly through the UI. Viewing and updating image metadata is also possible in this step. Following upload, the SHARKLINQ MODEL processes the image, generating and caching embeddings and match suggestions for the uploaded images. All newly uploaded images are initialized as unique shark entries which can be viewed, edited, and queried for matches.

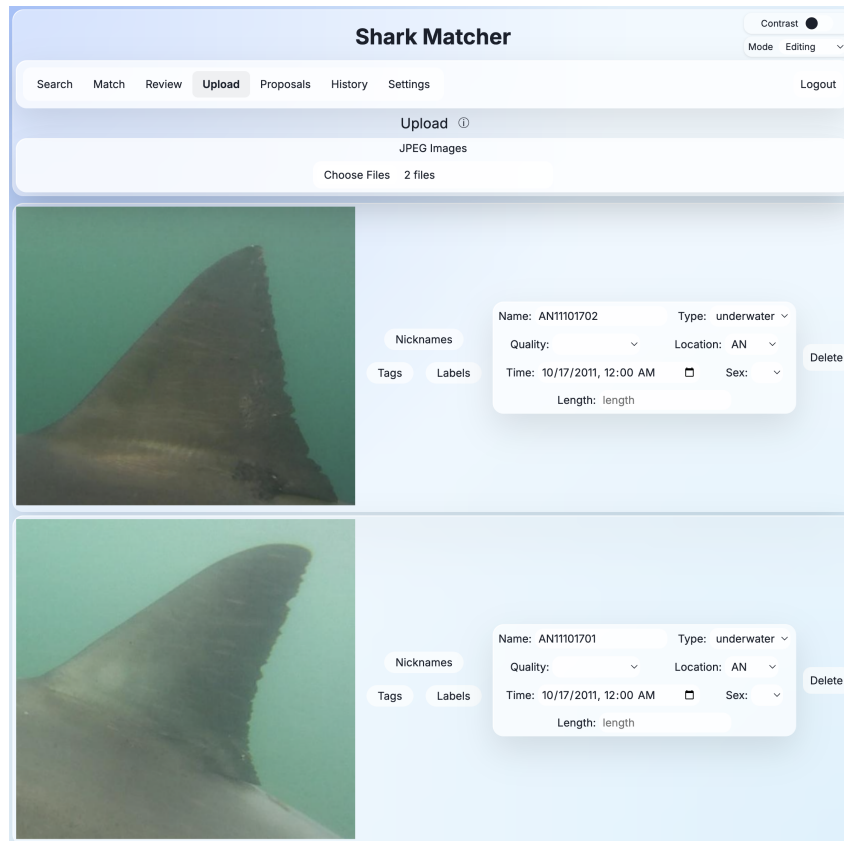


Figure 10. The image upload page during a bulk upload of images.

Search SHARK MATCHER’S search functionalities allow querying by sharks, images, and associated objects or filtering results by various constraints. Sharks can be queried by name or keywords including the name of any associated image or object. Filters can be combined with queries or used independently to limit results based on shark properties such as sex, length, or the number of associated images. The location filter allows filtering to sharks that have been encountered at a particular location. Only sharks with at least one image from the filtered location are returned. Similarly, minimum and maximum date filters restrict results to sharks whose images fall within a certain temporal range. A minimum date of January 1, 2020 filters to sharks that have at least one associated image after that date. Filters can also target associated objects such as returning only sharks with a “fresh” label or those carrying a tag. Critical in the context of wildlife data where metadata is often incomplete is the allow unknown filter. This filter explicitly includes or

excludes entries with unlabeled properties. For example, when filtering based on lengths of greater than 10 feet, sharks whose length is unknown can either be explicitly excluded or included with this setting. Search results are grouped by shark entry with each result displaying a shark’s profile information and full gallery of associated images (Figure 11).

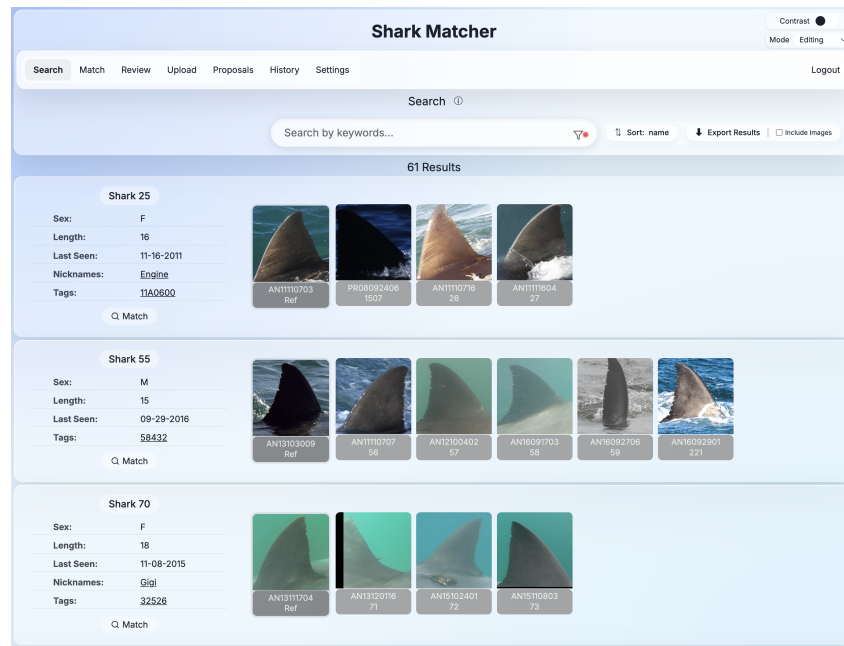


Figure 11. The search page shows results grouped by shark entry with key properties and an image gallery for each result.

Matching To review match suggestions for a shark, a shark or specific image is queried. When a shark entry is queried, the model defaults to using the entry’s reference image as the query to match against. Any of the shark entry’s other associated images can also be used as the matching query. Depending on image variance and the retrieval method specified, match suggestions across associated images within the same shark entry can differ based on the image queried.

SHARK MATCHER shows match suggestions grouped by shark entry, displaying the headline properties and full gallery of associated images for both the query and each match suggestion (Figure 12). Match results are divided into two columns.

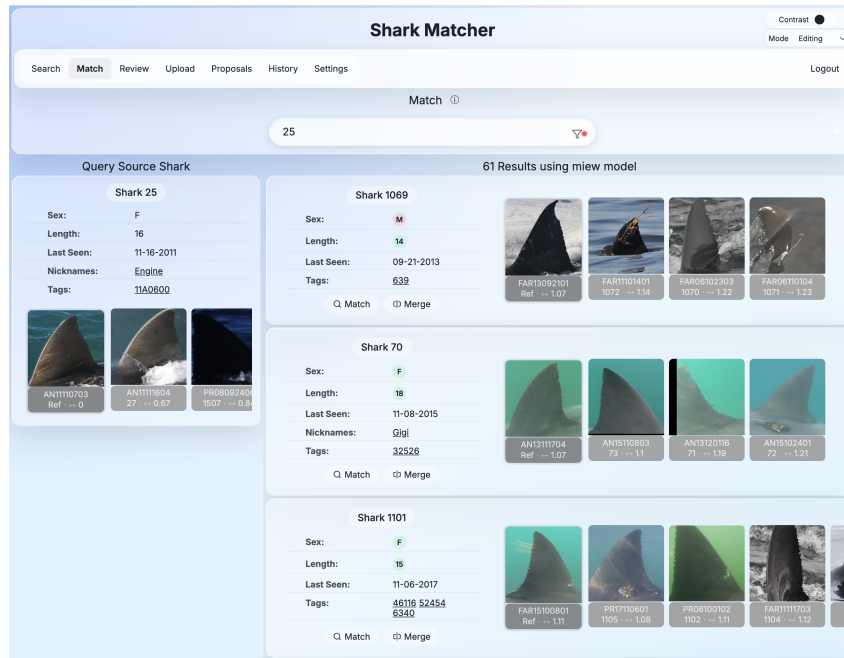


Figure 12. The match page displays the query shark (left) and an ordered list of match suggestions (right).

Left is the query column, showing the queried shark with all properties and associated images. In the associated image gallery, each image includes a distance value. This value quantifies similarity to the query image. The right column shows a ranked list of suggested matching shark entries. This list, ordered by distance scores, includes the shark’s profile and a scrollable gallery of associated images, also ordered by increasing distance, for each result. The match page layout is optimized for visual comparison and verification needed to confirm a match. A side-by-side “focus” view enlarges images from the query and suggested match. Arrow keys and shortcut keys allow viewers to flip images to check similarity. Image recolor filters can increase contrast to emphasize discriminative traits of the fin.

Merging Once a match between two shark entries is identified, they can be merged to group all their associated images and objects under a common shark entry. A merge preview page (Figure 13) shows the two incoming shark entries and a preview of the resulting merged result. For the resulting shark, the properties and asso-

ciated objects of the two inputs are automatically combined by SHARK MATCHER but can be manually overridden, including updating specific properties or adding and removing associated nicknames, tags, and labels.

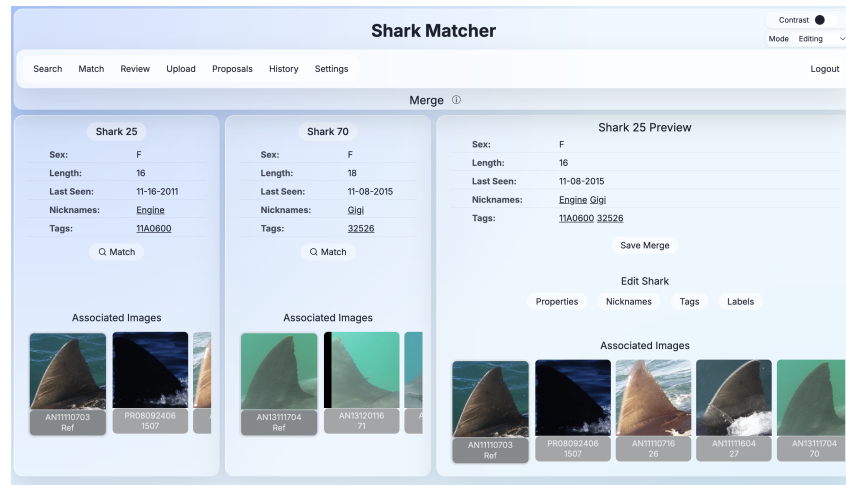


Figure 13. The merge page displays columns for the sharks to be merged (left) and a preview of the result (right).

Proposals SHARK MATCHER includes a tiered user system with different permission sets for viewer, collaborator, and admin users. This allows researchers to invite additional contributors including students, interns, or external collaborators with guardrails on write permissions. SHARK MATCHER has both direct editing and a proposal system, conceptually similar to pull requests in software engineering. Collaborators can make proposals but not directly mutate the data catalog. Instead, their actions, such as proposing a new match, correcting an animal’s length, or identifying an associated object, generate a proposal that can be reviewed and completed by admin users. Reviewers can search for, filter, view, evaluate, and either complete or archive proposals.

History To assure data is not lost or corrupted, every write operation within SHARK MATCHER is tracked providing a complete historical audit trail for every action. Actions can be searched and filtered via SHARK MATCHER’S history

tab. Displayed actions show the full details of the affected objects including a side-by-side comparison of the object’s state before and after the action. SHARK MATCHER also includes export functionalities to generate and save CSV snapshots of the database, allowing researchers to manage their data in SHARK MATCHER and export for further analysis, research, and sharing.

5.2 Impact

SHARK MATCHER has accelerated data curation for our collaborators at the Block Lab while also helping improve overall dataset quality, as highlighted by the evolution between our INITIAL and FULL data snapshots summarized in Table 11. Improved match suggestions and ease of use have eliminated the cycle of iteratively matching against multiple data partitions by making it feasible to match new images against the full catalog in a single pass. This has eliminated the need for temporary seasonal data partitions, simplifying ReID and reducing human error. Match suggestions have meaningfully reduced the average time to identify the shark in a new image. This acceleration comes from heavily reduced search effort to find potential matches and a smaller number of manual visual comparisons to verify the matching shark. Where previously the search space of sharks was a constantly growing pool which researchers had to navigate manually, our best performing model presents the correct matching shark within the first 5 match suggestions in 96% of queries. In the case of the Block Lab’s dataset, containing in excess of 1000 known unique sharks, this has significantly reduced the candidate pool for each match. Match suggestions have therefore shifted data labeling from a labor intensive search task to a quicker verification task where researchers review and validate high likelihood match suggestions.

Additionally, by pointing out potential mislabels and duplicate shark entries using the model’s embedding data, a review functionality has allowed the Block Lab to more easily identify and address potential data issues. The Block Lab has retroactively identified and merged 361 duplicate entries, cases where a single shark had two or more disjoint entries and identified 20 images that were associated with

an incorrect shark. These improvements in dataset quality have not only yielded more accurate downstream research, but have also provided a much cleaner ground-truth dataset for subsequent iterations of machine learning model training, reflected in the improved performance of full snapshot trained models over those trained on the initial snapshot.

6 Conclusion

In this work, we presented SHARKLINQ, an automated ReID system integrating a machine learning framework with a database management tool, built in close collaboration with marine biologists at Stanford University’s Block Lab. We detail key aspects of SHARKLINQ’s model framework and the SHARK MATCHER UI and present results on both ReID accuracy and the real-world impact of how marine biologists have been able to improve labeling efficiency and dataset quality. Our model framework directly addresses challenges inherent to many wildlife datasets, including fine-grained classes with long-tailed distributions, confounding factors including environment and image quality, and the possibility of incomplete or mislabeled data. We detail our research on model architecture, fine-tuning, multimodal representations, and retrieval techniques, and quantify their effects. A key feature of our model framework is its approach to combine a fine-tuned image model with metadata feature representations to leverage all facets of available data for improved ReID accuracy. Our evaluations demonstrate and break down the performance increase each of our presented approaches provides. Our fine-tuned MiewID image model paired with MULTIMODAL EMBEDDINGS and NEAREST PROTOTYPE retrieval achieves the strongest results across evaluations, scoring 90% Hits@1 retrieval accuracy, a mean average precision of 0.924, and finding the correct individuals within the first 50 match suggestions in all but 8 of the 842 queries in our SHARK REMOVAL test set.

Paired with SHARK MATCHER, the SHARKLINQ MODEL has provided meaningful benefit to the Block Lab’s researchers, significantly reducing the time they must invest in data processing and helping them improve dataset quality by locating and fixing labeling errors. Since the release of SHARK MATCHER, the Block Lab has enrolled a total of nine users from both their own and two partnered labs to collaborate on their dataset. Additional contributors uploading images and updating matches has had numerous benefits including greater data visibility, the opportunity to build a larger sharkfin dataset, and accelerated matching with more help from junior students, interns, and external collaborating researchers. Additional

labs with which the Block Lab has shared SHARK MATCHER have seen the impact and expressed interest in applying it to their own ReID and database management.

By open sourcing SHARKLINQ, including both the model framework and SHARK MATCHER, providing containerized runtime environments, and publishing extensive documentation, we hope to support additional labs seeking to streamline their ReID and data curation workflows. As the global community continues to amass increasingly large and diverse datasets of individually identified animals, the need for reliable, principled, and scalable tools to manage and leverage these resources becomes ever more critical. SHARKLINQ’s model framework and SHARK MATCHER represent a step toward meeting this need by combining numerous machine learning advancements with domain-specific considerations to ensure both performance and usability.

6.1 Future Work

There are numerous avenues for future work to expand on the foundations presented in this thesis, both to improve the model framework and SHARK MATCHER as a tool. One key area of interest is improving explainability of the embedding model through Class Activation Mapping (CAM) methods [34, 45], which produce color-coded localization maps that reflect the importance of specific regions in the image that most informed the final embedding. Not only could this inform future training optimizations, but it could also be integrated into SHARK MATCHER’S UI as an aid to the human-in-the-loop review process. Similarly, explainability techniques could be added to interpret and display multimodal matching weights and impacts given both MULTIMODAL EMBEDDINGS and SYMBOLIC RE-RANKING approaches are built on inherently interpretable feature weighting and constraint satisfaction mechanisms.

While our presented MULTIMODAL EMBEDDINGS technique effectively integrates ecological metadata with visual representations, several opportunities to expand this approach remain. Metadata representations could be optimized to better capture the continuous and asymmetric nature of biological data. This includes

replacing discrete location categories with geographic distance kernels and modeling temporal drift through asymmetric time-gap penalties. To leverage more complex cross-field dependencies, future work could also explore the use of foundation models to encode the entire metadata record as natural language prior to fusion, as proposed by Li et al. in MetaWild [23]. Embeddings for descriptive sentences (e.g., “Adult male whale shark photographed at Año Nuevo (AN) in August”), can naturally capture the semantic relationships between a subject’s location, season, and physical traits that flat feature vectors might miss.

Another area of interest is expanding SYMBOLIC RE-RANKING by combining it with our more successful NEAREST PROTOTYPE approach, which also uses k-reciprocal encoding. We have not yet combined NP retrieval with SYMBOLIC RE-RANKING since k-reciprocal encoding computes and compares both image embedding and Jaccard distances. Combining these methods requires further development since the SYMBOLIC RE-RANKING methodology also replaces pure embedding distances with scaled similarity truth values. Alternatively, a more unified approach could directly encode the mutual nearest neighbor principles of k-reciprocal encoding as additional logical constraints to be applied by SYMBOLIC RE-RANKING. This would allow the model to simultaneously reconcile structural visual similarities with the metadata constraints already being enforced.

Another future option is developing robust techniques for training under noisy data, a widespread issue in small-scale wildlife datasets. Our evaluations have shown the extent to which mislabeled and low quality data negatively impacts training and model performance. Integrating robust training techniques such as bounded loss functions to prevent the impact of mislabeled data [5, 20], sample selection to identify potential mislabels [16], and loss correction, which attempts to model and correct noise during training [28], may help more reliably tune a high performing ReID model, which can then be applied to correct labeling errors. To this end, we hope to also expand SHARK MATCHER’S UI with a match recommendation page that can proactively point out potential matches between sharks and flag incorrectly labeled images.

Extending our work by fine-tuning additional image foundation models on real-

world datasets with different species can help better understand the model framework's generalization potential. Similarly, different datasets can contain new metadata features with variable levels of accuracy and frequency, allowing more comprehensive analysis of our multimodal approaches. When working across datasets, the metadata ingestion pipeline and representations can be adjusted and weights can be re-trained to suit the available data, making this a transferable technique. Lastly, more widespread use of SHARK MATCHER as a generalized wildlife image labeling tool will allow its layout and interface to be tested across species and workflows, prompting features to be added and adapted to expand functionality.

Bibliography

- [1] Scot Anderson, Taylor Chapple, Salvador Jorgensen, A. Klimley, and Barbara Block. Long-term individual identification and site fidelity of white sharks, *carcharodon carcharias*, off california using dorsal fins. *Marine Biology*, 2011.
- [2] Samantha Andrzejaczek, Taylor K. Chapple, Salvador J. Jorgensen, Scot D. Anderson, Michael Castleton, Paul E. Kanive, Timothy D. White, and Barbara A. Block. Multi-decadal high-resolution data reveal the cryptic vertical movement patterns of a large marine predator along the californian coast. *Frontiers in Marine Science*, 2022.
- [3] Stephen Bach, Matthias Broecheler, Bert Huang, and Lise Getoor. Hinge-loss markov random fields and probabilistic soft logic. *Journal of Machine Learning Research (JMLR)*, 2017.
- [4] Jay Barlow, John Calambokidis, Erin A Falcone, C Scott Baker, Alexander M Burdin, Phillip J Clapham, John KB Ford, Christine M Gabriele, Richard LeDuc, David K Mattila, et al. Humpback whale abundance in the north pacific estimated by photographic capture-recapture with bias correction from simulation studies. *Marine Mammal Science*, 2011.
- [5] Jonathan T. Barron. A general and adaptive robust loss function. *arXiv*, 2019.
- [6] Tanya Berger-Wolf, Daniel Rubenstein, Charles Stewart, Jason Holmberg, Jason Parham, Sreejith Menon, J.P. Crall, Jon Van Oast, Emre Kiciman, and Lucas Joppa. Wildbook: Crowdsourcing, computer vision, and data science for conservation. *arxiv*, 2017.
- [7] Christian Bergler, Alexander Gebhard, Jared R. Towers, Leonid Butyrev, Gary J. Sutton, Tasli J. H. Shaw, Andreas Maier, and Elmar Nöth. Fin-print a

- fully-automated multi-stage deep-learning-based framework for the individual recognition of killer whales. *Nature Scientific Reports*, 2021.
- [8] Drew Blount, Shane Gero, Jon Van Oast, Jason Parham, Colin Kingen, Ben Scheiner, Tanya Stere, Mark Fisher, Gianna Minton, Christin Khan, Violaine Dulau, Jaime Thompson, Olga Moskvayak, Tanya Berger-Wolf, Charles Stewart, and Jason Holmberg. Flukebook: An open-source ai platform for cetacean photo identification. *Mammalian Biology*, 2022.
 - [9] Cuiqun Chen, Mang Ye, and Ding Jiang. Towards modality-agnostic person re-identification with descriptive query. In *cvpr*, 2023.
 - [10] Jonathan P. Crall, Charles V. Stewart, Tanya Y. Berger-Wolf, Daniel I. Rubenstein, and Siva R. Sundaresan. Hotspotter — patterned species instance recognition. In *IEEE Workshop on Applications of Computer Vision (WACV)*, 2013.
 - [11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *cvpr*, 2009.
 - [12] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. Arcface: Additive angular margin loss for deep face recognition. In *CVPR*, 2019.
 - [13] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv*, 2021.
 - [14] Kuniyuki Fukushima, Sei Miyake, and Takayuki Ito. Neocognitron: A neural network model for a mechanism of visual pattern recognition. *IEEE Transactions on Systems, Man, and Cybernetics*, 1983.
 - [15] Philip Hammond, Tessa Francis, Dennis Heinemann, Kristy Long, Jeffrey Moore, André Punt, Randall Reeves, Maritza Sepulveda, Gudjon Sigurdsson, Margaret Siple, Gísli Víkingsson, Paul Wade, Rob Williams, and Alexandre Zerbini. Estimating the abundance of marine mammal populations. *Frontiers in Marine Science*, 2021.
 - [16] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor W.

- Tsang, and Masashi Sugiyama. Co-teaching: robust training of deep neural networks with extremely noisy labels. In *NeurIPS*, 2018.
- [17] G.R. Hillman, N. Kehtarnavaz, Bernd Würsig, Babak Araabi, Glenn Gailey, David Weller, S. Mandava, and H. Tagare. Finscan, a computer system for photographic identification of marine animals. In *IEEE Biomedical Engineering*, 2002.
 - [18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv*, 2021.
 - [19] Benjamin Hughes and Tilo Burghardt. Automated visual fin identification of individual great white sharks. *arXiv*, 2016.
 - [20] Lu Jiang, Zhengyuan Zhou, Thomas Leung, Li-Jia Li, and Li Fei-Fei. MENTORNET: Learning data-driven curriculum for very deep neural networks on corrupted labels, 2018.
 - [21] Ziyang Jiang, Rui Meng, Xinyi Yang, Semih Yavuz, Yingbo Zhou, and Wenhui Chen. VLM2VEC: Training vision-language models for massive multimodal embedding tasks. In *International Conference on Learning Representations*, 2025.
 - [22] Siyuan Li, Li Sun, and Qingli Li. Clip-reid: exploiting vision-language model for image re-identification without concrete text labels. In *AAAI*, 2023.
 - [23] Yuzhuo Li, Di Zhao, Tingrui Qiao, Yihao Wu, Bo Pang, and Yun Sing Koh. Metawild: A multimodal dataset for animal re-identification with environmental metadata. In *acm*, 2025.
 - [24] Ibrahim Majid Mohammed and Nor Ashidi Mat Isa. Contrast limited adaptive local histogram equalization method for poor contrast image enhancement. *IEEE Access*, 2025.
 - [25] Megan Maroen, Mary Rowlinson, Conrad Matthee, and Sara Andreotti. The effectiveness of the semi-automated IDENTIFIN software to accurately fast-track dorsal fin photographic identifications of sharks. *Aquatic Conservation: Marine and Freshwater Ecosystems*, 2024.
 - [26] Olga Moskvayak, Frederic Maire, Asia O. Armstrong, Feras Dayoub, and

- Mahsa Baktashmotlagh. Robust re-identification of manta rays from natural markings by learning pose invariant embeddings. In *Digital Image Computing: Techniques and Applications (DICTA)*, 2021.
- [27] Ekaterina Nepovinnikh, Ilia Chelak, Tuomas Eerola, Veikka Immonen, Heikki Kälviäinen, Maksim Kholiavchenko, and Charles Stewart. Species-agnostic patterned animal re-identification by aggregating deep local features. *International Journal of Computer Vision*, 2024.
- [28] Curtis Northcutt, Lu Jiang, and Isaac Chuang. Confident learning: Estimating uncertainty in dataset labels. *J. Artif. Int. Res.*, 2021.
- [29] Douglas Nowacek, Fredrik Christiansen, Lars Bejder, Jeremy Goldbogen, and Ari Friedlaender. Studying cetacean behaviour: new technological approaches and conservation applications. *Animal Behaviour*, 2016.
- [30] Lasha Otarashvili, Tamilselvan Subramanian, Jason Holmberg, J. J. Levenson, and Charles V. Stewart. Multispecies animal re-id using a large community-curated dataset. *arXiv*, 2024.
- [31] Jason Parham, Jonathan Crall, Daniel Rubenstein, Jason Holmberg, Tanya Berger-Wolf, , and Charles Stewart. An animal detection pipeline for identification. *International Journal of Modern Physics B*, 2018.
- [32] J. J. Rocchio. Relevance feedback in information retrieval. In *The Smart retrieval system - experiments in automatic document processing*, 1971.
- [33] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [34] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*, 2019.
- [35] Mingxing Tan and Quoc V. Le. Efficientnetv2: Smaller models and faster training. In *International Conference on Machine Learning*, 2021.
- [36] Jaime Thompson, Victoria Zero, Lori Schwacke, Todd Speakman, Brian Quigley, Jeanine Morey, and Trent McDonald. finfindr : Automated recog-

- nition and identification of marine mammal dorsal fins using residual convolutional neural networks. *Marine Mammal Science*, 2021.
- [37] Anna Van Tienhoven, J. Hartog, Renate Reijns, and Victor Peddemors. A computer-aided program for pattern-matching of natural marks on the spotted raggedtooth shark *carcharias taurus*. *Journal of Applied Ecology*, 2007.
 - [38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
 - [39] Hendrik Weideman, Zachary Jablons, Jason Holmberg, Kiirsten Flynn, John Calambokidis, Reny Tyson Moore, Jason Allen, Randall Wells, Krista Hupman, Kim Urian, and Charles Stewart. Integral curvature representation and matching algorithms for identification of dolphins and whales. In *ICCV*, 2017.
 - [40] Randall S Wells and Michael D Scott. Estimating bottlenose dolphin population parameters from individual identification and capture-release techniques. *Reports of the International Whaling Commission*, 1990.
 - [41] Bernd Wuersig and T.A. Jefferson. Methods of photo-identification for small cetaceans. *Methods of Photo-identification for Small Cetaceans*, 1990.
 - [42] Zhaoxiang Zheng, Yaqin Zhao, Ao Li, and Qiuping Yu. Wild terrestrial animal re-identification based on an improved locally aware transformer with a cross-attention mechanism. *Animals*, 2022.
 - [43] Zhun Zhong, Liang Zheng, Donglin Cao, and Shaozi Li. Re-ranking person re-identification with k-reciprocal encoding. In *cvpr*, 2017.
 - [44] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *AAAI 2017*, 2017.
 - [45] Bolei Zhou, Aditya Khosla, Àgata Lapedriza, Aude Oliva, and Antonio Torralba. Learning deep features for discriminative localization. In *Conference on Computer Vision and Pattern Recognition*, 2016.
 - [46] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Learning to prompt for vision-language models. *International Journal of Computer Vision*, 2022.